CET S5 ELECTRICAL AND COMPUTER ENGINEERING

Modern Operating Systems

Modules 1 and 2

PEEOT521 • Exam study guide

From first principles to exam answers

Operating system functions and booting. Processes, threads and IPC. CPU scheduling with fully worked numerical examples. Critical sections, synchronization tools and the three classical synchronization problems.
Expanded self-study edition: short introductory examples, step-by-step traces and a topic-by-topic GfG comparison.
Every named topic in the two syllabus modules is mapped to a section. Pseudocode is explained, assumptions are stated, and common mistakes are corrected.

Reading and print edition

Prepared against the local Electrical and Computer Engineering syllabus. Checked 7 October 2026.

Contents

Scope, sources and how to study 1

I Module 1: Operating systems, processes and IPC 2

1 Operating system overview and functions 2
1.1 What an operating system does . . . 2
1.2 Functions: connect each service to an example . . . 2
1.3 Execution control: modes, interrupts and system calls . . . 3
1.4 Supporting vocabulary: ways systems organise work . . . 4
1.5 Follow one ordinary task through the OS . . . 4
1.6 Classify systems by the problem they solve . . . 4
1.7 A service request, step by step . . . 5
1.8 Kernel organisation: supporting overview . . . 5

2 Boot process 6
2.1 From power-on to a usable system . . . 6
2.2 Understand the stages rather than memorising vendor names . . . 7

3 Processes: states, queues and the PCB 7
3.1 Program versus process . . . 7
3.2 Five-state process model . . . 7
3.3 Process control block . . . 8
3.4 Context switching . . . 8
3.5 A process-state trace you can reason through . . . 9
3.6 Process table versus a single PCB . . . 9
3.7 Suspension: distinguish two independent questions . . . 9

4 Threads and multithreading 10
4.1 What is shared and what is private? . . . 10
4.2 Benefits and limits . . . 10
4.3 User and kernel threads; mapping models . . . 10
4.4 Choose processes or threads deliberately . . . 11
4.5 Visualise the mapping models . . . 12

5 Scheduling in the process life cycle 12
5.1 Queues and schedulers . . . 12

6 Process creation and termination 13
6.1 Creation: identity, resources and execution . . . 13
6.2 Unix-style fork, exec, wait and exit . . . 13
6.3 Termination and cleanup . . . 14
6.4 Track the address spaces after creation . . . 14

7 Inter-process communication 15
7.1 Why processes cooperate . . . 15
7.2 Shared-memory systems . . . 15
7.3 Message-passing systems . . . 16
7.4 Shared memory versus message passing . . . 17

7.5 Separate data transfer from the communication protocol . . . 17
7.6 Understand the two-index circular buffer . . . 17
7.7 Which mechanism fits which communication? . . . 18

II Module 2: Scheduling and synchronization 19

8 CPU scheduling: basic concepts and criteria 19
8.1 CPU–I/O burst cycle . . . 19
8.2 Preemptive and nonpreemptive scheduling . . . 19
8.3 Scheduling criteria . . . 20
8.4 The formulas and their assumptions . . . 20
8.5 A reliable numerical method . . . 20
8.6 Read a CPU burst as service, not elapsed lifetime . . . 21
8.7 The common event loop behind the policies . . . 21

9 Common data set for the scheduling algorithms 21

10 First Come First Served (FCFS) 22
10.1 Build FCFS by maintaining the queue . . . 22

11 Shortest Job First (SJF) and its preemptive variant 23
11.1 Nonpreemptive SJF . . . 23
11.2 Shortest Remaining Time First (SRTF) . . . 23
11.3 Why short-before-long reduces average waiting . . . 24
11.4 Distinguish estimation from selection . . . 24

12 Priority scheduling 24
12.1 Nonpreemptive worked example . . . 25
12.2 Preemptive worked example . . . 25
12.3 Starvation, ageing and supporting terminology . . . 25
12.4 Priority changes must have a defined meaning . . . 26
12.5 A priority-inversion timeline . . . 26

13 Round Robin (RR) 26
13.1 Queue method: quantum 3 . . . 26
13.2 Choosing the quantum . . . 27
13.3 Write the queue beside the Gantt chart . . . 27

14 Comparison and numerical cross-checks 28

15 Race conditions and the critical-section problem 28
15.1 Why interleaving can change a result . . . 28
15.2 Critical sections and three correctness requirements . . . 29
15.3 Two failed software approaches . . . 30
15.4 What must be protected? . . . 30

16 Peterson's solution 30
16.1 Two-process algorithm and assumptions . . . 30
16.2 Walk through simultaneous interest . . . 31
16.3 Why it satisfies the three requirements . . . 31

16.4 Limits and implementation caution 31

17 Synchronization hardware 32
17.1 Disabling interrupts: scope and limits 32
17.2 Atomic test-and-set 32
17.3 Atomic compare-and-swap (CAS) 32
17.4 Adding bounded waiting: an explicit handoff protocol 33
17.5 Atomic exchange/swap: another hardware building block 33

18 Mutex locks 34
18.1 Spinning versus blocking 34
18.2 Ownership and correctness 34

19 Semaphores 35
19.1 Atomic wait and signal 35
19.2 Mutual exclusion and ordering are different uses 35
19.3 Blocking implementation and the negative-count convention 35
19.4 Mutex versus semaphore 36
19.5 Typical errors and liveness issues 36
19.6 Trace the two semaphore count conventions 36
19.7 Choose the abstraction by the required condition 37

20 Monitors and condition variables 37
20.1 Monitor structure 37
20.2 Condition wait and signal 38
20.3 Hoare versus Mesa semantics; why a while loop is used 38
20.4 Condition variable versus semaphore 38
20.5 A monitor that combines exclusion and a predicate 39

21 Producer–consumer: the bounded-buffer problem 39
21.1 Problem and requirements 39
21.2 Semaphore solution: complete initialisation and code 39
21.3 Worked count trace and the in-flight nuance 40
21.4 The dangerous reversed wait order 41
21.5 Monitor solution 41

22 Readers–writers problem 42
22.1 Problem and policy choices 42
22.2 Reader-preference semaphore solution 42
22.3 A fair-admission turnstile solution 43
22.4 Writer-preference monitor variant 44

23 Dining philosophers 45
23.1 Problem model and the naive failure 45
23.2 Solution 1: admit at most four contenders 46
23.3 Solution 2: globally order the forks 46
23.4 Solution 3: monitor and neighbour state tests 47
23.5 Recognise the three different resource patterns 48
23.6 Why a correct-looking solution may still starve someone 48

24 Exam revision: concise answers 49

24.1 Module 1: short-answer practice . . . 49
24.2 Module 2: short-answer practice . . . 50

25 Long-answer practice and worked solutions 53
25.1 OS functions and the complete boot sequence . . . 53
25.2 Process states, PCB and context switching . . . 53
25.3 Process operations and threads . . . 53
25.4 Both IPC models and message semantics . . . 53
25.5 Fresh numerical problem: FCFS, SJF and RR . . . 54
25.6 Peterson's solution with correctness argument . . . 55
25.7 Hardware locks, mutexes and semaphores . . . 55
25.8 Bounded buffer and a faulty wait order . . . 55
25.9 Readers–writers and policy . . . 55
25.10 Dining philosophers with two solutions . . . 55
25.11 Semaphore ordering and a Mesa wait trace . . . 55

26 Final syllabus checklist and common mistakes 55
26.1 Check these after active recall . . . 56
26.2 Mistakes that change the answer . . . 56

27 Sources and recommended study resources 57
27.1 Use a small, purposeful set of resources . . . 57
27.2 OSTEP: free extended explanations . . . 57
27.3 University notes: compact second explanations . . . 58
27.4 Complementary textbook and practice websites . . . 59
27.5 Video options . . . 59
27.6 Reproducible verification record . . . 59

28 GeeksforGeeks topic map and expanded lessons 60
28.1 Module 1: topic links . . . 60
28.2 Module 2: topic links . . . 60

Scope, sources and how to study

The controlling syllabus is **PEEOT521: Modern Operating System** in the supplied S3–S8 Electrical and Computer Engineering syllabus, physical PDF pages 171–174. The subject code on some older local notes is CST305; use the PEEOT521 coverage below for this course.

Module	Exact topic groups in the syllabus	Guide location
1: 8 hours	OS overview, functions, boot process	1–2
1	Process states, PCB, threads, scheduling	3–5
1	Process creation and termination	6
1	IPC: shared memory and message passing	7
2: 10 hours	Scheduling basics, criteria; FCFS, SJF, priority, round robin	8–14
2	Race conditions, critical section, Peterson's solution	15–16
2	Synchronization hardware, mutex locks, semaphores, monitors	17–20
2	Producer–consumer, dining philosophers, readers–writers	21–23

The exam structure in this syllabus: 60 marks in 2 hours 30 minutes. Part A has two compulsory 3-mark questions from each of four modules. In Part B, choose one of two full 9-mark questions per module; each can have up to three subdivisions. Each module therefore accounts for 15 marks. Practise both concise definitions and explanations with diagrams, code or calculations.

Reading order. First read Module 1 and draw the process-state diagram from memory. Next solve the scheduling examples without looking at the answers. Then learn the critical-section requirements and each synchronization primitive before attempting the classical problems. Finish with the revision questions and the syllabus checklist.

Depth and boundaries. SRTF, CPU-burst prediction, context switching, thread models and fair readers–writers help explain named syllabus topics. They are marked as supporting detail where appropriate. Deadlock and memory/file/storage management are covered in the separate Modules 3–4 guide; this guide explains only the connections necessary for Modules 1–2.

Verification. Official *Operating System Concepts*, ninth-edition slides, university course notes and the author-hosted *Operating Systems: Three Easy Pieces* (OSTEP) were used to check concepts. The main scheduling result tables are generated by a reproducible script and checked against separately specified timelines. New introductory calculations and queue states are checked independently. Finite-state checks exercise Peterson, the bounded buffer and a dining solution under the stated assumptions. These checks supplement the explanations; they do not establish real-machine correctness or starvation freedom. The final resource section gives chapter mappings and links.

How to use pseudocode

`wait(S)` and `signal(S)` denote atomic semaphore operations; `cv.wait()` denotes a monitor condition wait, not a semaphore wait. Procedures shown inside a monitor execute with its implicit lock. Code is language-neutral teaching pseudocode, unless explicitly identified as an API example. Plain shared variables in C/C++ do not automatically have the atomicity or ordering required by a textbook algorithm.

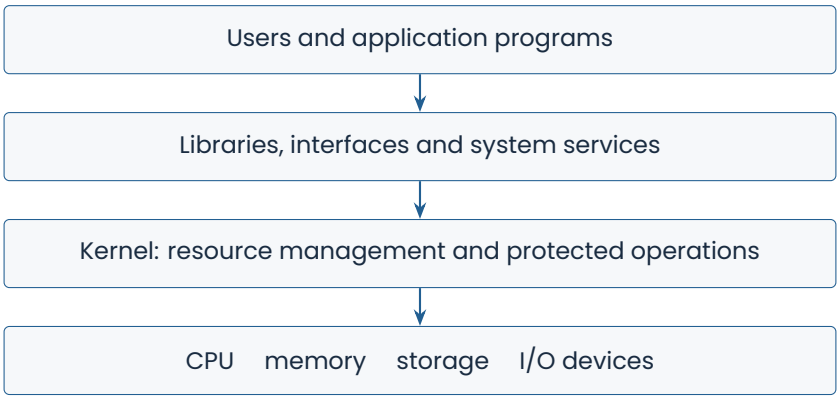
I Module 1: Operating systems, processes and IPC

1 Operating system overview and functions

1.1 What an operating system does

An **operating system (OS)** manages a computer's hardware resources and provides services through which programs can use those resources. It sits between applications and hardware. Instead of requiring each program to control a disk or implement CPU sharing, the OS provides abstractions such as files, processes and virtual address spaces.

Two useful views are complementary. As a **resource manager**, the OS allocates CPU time, memory, storage and devices, and resolves competing requests. As a **control program**, it supervises execution, handles errors and prevents one program from improperly interfering with another.



The **kernel** is the central privileged component of the OS. Shells, graphical interfaces, utilities and many services may run outside it. A shell is a command interpreter; it is not the kernel. The precise boundary of what a vendor calls the operating system varies.

1.2 Functions: connect each service to an example

Function	Meaning and example
Program execution	Load a program, establish its execution environment, run it and handle normal or abnormal termination.
Process and CPU management	Create/terminate processes, schedule CPU work, maintain process state and support synchronization and communication.
Memory management	Track memory use, allocate/reclaim memory and provide protected address spaces. Virtual memory lets a program use an address space different from physical RAM.
File-system management	Create, open, read, write and delete files/directories; maintain metadata and permissions. A file abstracts persistent storage.
Device and I/O management	Drivers translate general operations into device-specific commands. The OS coordinates interrupts, transfers and device access.

Function	Meaning and example
Secondary-storage management	Track free space, allocate storage and schedule requests. Persistent storage outlives a process.
Communication	Let processes exchange information locally or across networks, using shared memory, messages and related services.
Protection and security	Enforce who can access which resources; authenticate identities and help defend the system. Protection concerns controlled access; security includes broader threats.
Error detection and recovery	Detect device failures, invalid memory accesses and other faults; report or handle them without silently corrupting unrelated work.
Resource allocation and accounting	Share resources among competing users/programs and measure consumption for monitoring, limits or billing.
User interface	Offer command-line, graphical or other interaction through which users request services.

Buffering, caching and spooling are different

Buffering temporarily holds data during transfer to handle speed or size differences: an input buffer between a keyboard and an application. **Caching** keeps a copy of data likely to be used again: recently read disk blocks in RAM. **Spooling** places jobs in a queue, often on storage, for a device that handles them sequentially: several print jobs waiting for one printer. A system may use all three together.

1.3 Execution control: modes, interrupts and system calls

User mode restricts privileged operations; **kernel mode** permits the operations required to control hardware and manage resources. Hardware tracks the current mode. User programs request protected services through **system calls**, such as creating a process or reading a file. A library function may wrap a system call, or may perform its work entirely in user space.

A system-call instruction transfers control to a defined kernel entry point. The kernel validates the request and arguments, performs the operation and eventually returns. **Interrupts** typically report asynchronous events, such as a timer expiry or completed device transfer. **Exceptions/traps** arise synchronously during instruction execution, such as an invalid operation or deliberate service request; terminology differs across architectures. Kernel entry can occur through these mechanisms as well as through a system call.

A **timer interrupt** helps prevent a process from monopolising the CPU: control periodically returns to the OS, which can schedule another runnable process. Entering kernel mode does not by itself mean a different process has been selected.

1.4 Supporting vocabulary: ways systems organise work

Term	Essential idea
Batch processing	Submit jobs for execution with little interaction while they run.
Multiprogramming	Keep several jobs in memory so another can run when the current job waits, improving CPU utilisation.
Multitasking/time sharing	Switch CPU attention frequently to support responsive interaction and concurrent activities.
Multiprocessing	Use multiple processors or cores; different tasks can actually execute in parallel.
Real-time system	Correctness involves timing constraints as well as the computed result. Hard deadlines must be met; soft deadlines tolerate some misses.

Concurrency means activities make progress during overlapping periods. **Parallelism** means simultaneous execution. A single CPU core can support concurrency by interleaving; multiple cores enable parallel execution.

Further reading: OSC 9e, Chapter 1; selected OS services and system-call material in Chapter 2.

1.5 Follow one ordinary task through the OS

Imagine opening a document while music is playing. The editor needs CPU time to decode the document, memory to hold its working data and storage access to retrieve bytes. The audio player also needs regular CPU service and a device output stream. The OS coordinates these requests so both programs can progress.

The editor sees a filename rather than a platter location; this is **abstraction**. Its private addresses do not normally let it overwrite the audio player's memory; this is **protection**. Choosing which runnable activity gets the CPU next is **scheduling**. Deciding who may open the document is an **access-control** decision. These are different OS functions cooperating in one task.

Convenience and efficiency can conflict. Giving each program a simple private view requires bookkeeping; letting every program control devices directly might reduce a layer of overhead but would lose controlled sharing. Good OS design provides useful abstractions while managing the cost of implementing them.

1.6 Classify systems by the problem they solve

The categories below describe design emphasis and can overlap. A system can be both multiprocessor and time-sharing; the labels are not mutually exclusive product bins.

Emphasis	Typical situation	What to remember
Batch	Many submitted jobs execute without continual user interaction	Throughput matters; an individual user may wait a long time for a result.
Multiprogramming	A computing job runs while another waits for I/O	Keep work available for idle resources. It does not require several cores.
Time sharing	Several users or interactive tasks need attention	Short turns improve responsiveness; switching has a cost.
Multiprocessing	Independent work can run on several cores	Actual parallel execution, coordinated access and load distribution.
Real time	A sensor/controller must act within a required time	A correct value delivered too late may be unacceptable. Fast average response alone is insufficient.
Network or distributed	Work or services involve connected machines	A network OS exposes remote services; a distributed design may coordinate machines behind a more unified view.
Embedded or mobile	Software serves a dedicated device or portable environment	Power, memory, device interaction and reliability shape decisions.

Network/distributed and embedded/mobile are supporting overview vocabulary. Focus your exam preparation on the actual functions and process mechanisms named in the syllabus.

1.7 A service request, step by step

A user-space program calls a library routine to read an open file. The routine arranges the system-call arguments. Hardware enters a defined kernel path; the kernel validates the request and obtains the bytes. If they are cached, it may finish promptly. If storage I/O is required, the caller may block and another ready task can run. The eventual result returns to the caller.

This explains three important distinctions: a library call need not itself be a system call; a system call need not always block; and kernel entry need not always switch to another process. Device completion uses an interrupt path rather than requiring the waiting program to keep checking every instant.

1.8 Kernel organisation: supporting overview

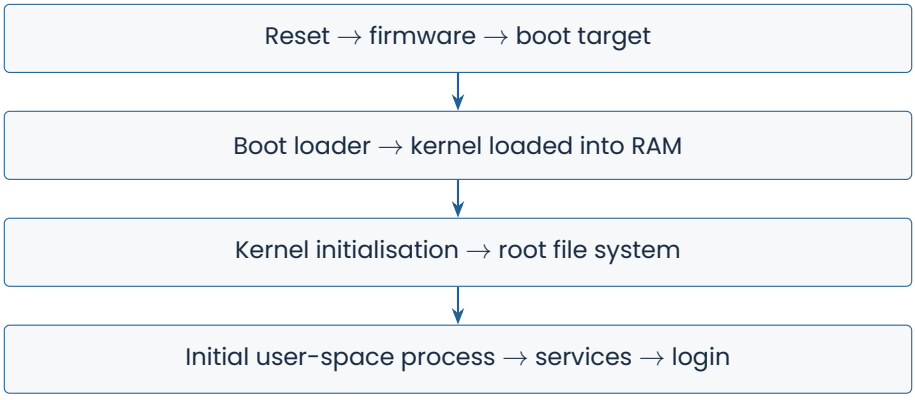
A **monolithic** kernel keeps many OS services in one privileged environment, making interaction direct but increasing the amount of trusted kernel code. A **microkernel** keeps a smaller set of mechanisms privileged and places more services in separate processes, with communication between them. Loadable modules and hybrid arrangements provide other practical choices. These terms describe organisation; they do not change the definitions of user mode, process or system call.

2 Boot process

2.1 From power-on to a usable system

Booting is the sequence that initialises hardware, brings the kernel into memory and starts the operating environment. A **bootstrap program** performs the initial steps and arranges for the kernel to run. The initial firmware is stored in nonvolatile memory; the full kernel is normally loaded from persistent storage, though network and embedded boot arrangements also exist.

1. **Power-on or reset.** The processor begins at an architecture-defined location and executes firmware.
2. **Firmware initialisation.** Firmware sets up essential hardware and performs checks sufficient to continue. On PCs, BIOS and UEFI are examples of firmware interfaces; they are not the OS itself.
3. **Select a boot target.** Firmware follows configured boot order or a user selection to identify storage, a network target or another boot source.
4. **Run a boot loader.** The loader locates and loads the kernel; it may present an OS menu and supply parameters and an initial RAM file system. Some systems use multiple loader stages; UEFI may also load a kernel through an EFI-compatible entry path.
5. **Kernel initialisation.** The kernel establishes memory-management structures, interrupt handling, scheduling and required device drivers. It discovers or initialises devices needed to proceed.
6. **Make the root file system available.** A temporary initial environment may load storage drivers and locate the persistent root file system before switching to it.
7. **Start the first user-space process and services.** In a Unix-like system, an init/service manager starts further services and sessions. A name such as `systemd` is an implementation example, not a universal boot stage.
8. **Present a usable interface.** A login prompt, graphical session or application becomes available; background services continue as configured.



A **cold boot** starts from power-off; a **warm boot** restarts an already powered system. The amount of hardware reinitialisation varies. Booting is different from merely starting another application after the OS is running.

2.2 Understand the stages rather than memorising vendor names

Symptom	Stage to investigate first
Firmware cannot detect boot storage	Hardware detection or firmware configuration.
A boot device is found but no loader starts	Boot-target configuration or loader installation.
Loader starts but cannot find a kernel	Loader path/configuration or missing kernel image.
Kernel starts but cannot mount root	Storage support, root-device identification or file-system availability.
Kernel runs but no login is available	Initial user-space process, service configuration or session startup.

These are diagnostic starting points, not unique proofs of the fault. For an exam answer, give the ordered sequence and explain the loader's role. Do not state that every OS necessarily uses BIOS, GRUB or a particular init program.

Further reading: OSC 9e, Chapter 1 bootstrap discussion and Chapter 2 system boot.

3 Processes: states, queues and the PCB

3.1 Program versus process

A **program** is a passive set of instructions stored in a file. A **process** is an executing instance with its own execution state and resources. Several processes can execute the same program without being the same process.

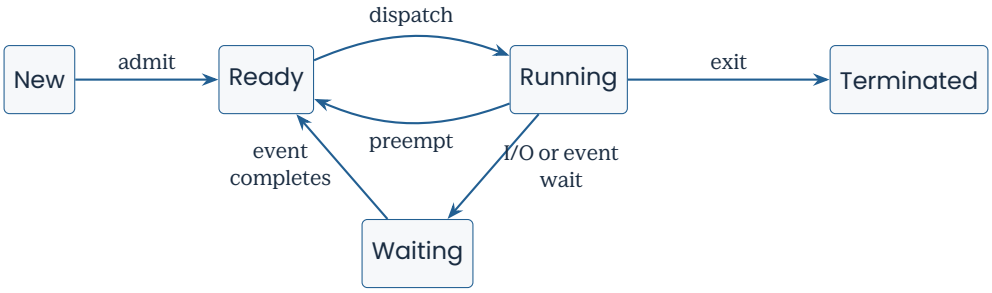
A typical process address space contains **text/code**, **data** for global/static variables, a **heap** for dynamic allocations and a **stack** for calls, local variables and return information. The current program counter and register contents describe execution at an instant. Exact layouts, growth directions and sharing depend on the implementation.

Worked example: Two instances of the same program

Opening two independent instances of an editor can create two processes with different PIDs, stacks, heaps and open documents. The OS may share read-only code pages, yet each process retains its own execution context and protected virtual address space. Shared program text does not imply that their writable data is automatically shared.

3.2 Five-state process model

State	Meaning
New	Being created; not yet admitted to the ready population in this model.
Ready	Able to run but waiting for CPU assignment.
Running	Executing on a CPU.
Waiting (blocked)	Unable to proceed until an event occurs, such as I/O completion or a synchronization condition.
Terminated	Execution has ended; final cleanup/accounting may still be needed.



The ready process already has everything required to execute except the CPU. A waiting process lacks a required event or condition, so selecting it would not allow useful progress.

An I/O completion normally moves Waiting to Ready, not directly to Running. The scheduler must still select that process. A timer expiry can move Running to Ready even though the process has not blocked. On a single CPU, at most one process/thread executes at a time; on a multicore machine several may be running.

Supporting extension: suspended states. Some models distinguish ready-suspended and blocked-suspended processes whose memory image has been moved out of main memory. Suspension concerns residency/control, while ready versus blocked concerns whether the process can proceed. These extra states do not replace the basic five-state model.

3.3 Process control block

The **process control block (PCB)** is the OS data structure that records information needed to manage a process and resume its execution. It is maintained by the OS, not simply the program's source code.

PCB information	Why it is needed
PID, related identifiers, state	Identify the process, its relationships and its current status.
Program counter and CPU registers	Restore the exact execution point and working values after interruption. Includes appropriate stack-pointer/context information.
CPU scheduling information	Priority, queue links and related scheduling attributes.
Memory-management information	Address-space description, page-table references or other mapping/protection information.
I/O and file information	Open-file references, allocated devices and pending I/O information.
Accounting and credentials	CPU usage, limits, ownership/security credentials and other management data.

The exact split between process-wide structures and per-thread control blocks varies. A thread's PC, registers and stack belong to that thread's context; an address space is usually process-wide.

3.4 Context switching

A **context switch** saves one execution context and restores another. Conceptually, the OS saves the old PC/registers and state, updates relevant queues, selects a ready task, restores its context

and resumes it. Address-space changes may be required when switching between processes; threads of the same process can share the address space.

Context switching consumes time without advancing the switched user tasks' application work. Cost depends on hardware, saved state, cache/TLB effects and the kinds of tasks involved. It is not a fixed universal number.

Mode switch versus context switch

A process may enter kernel mode to perform a system call and return to the same process without a process context switch. Conversely, a scheduling event may switch to a different process. Distinguish the privilege level from the identity of the running task.

Further reading: OSC 9e, Chapter 3; OSTEP Chapters 4–5 for process intuition and APIs.

3.5 A process-state trace you can reason through

Worked example: A document reader and a calculation

Process A is admitted and becomes Ready. Dispatch gives it the CPU, so it is Running. It requests unavailable storage data and becomes Waiting. B, already Ready, runs next. Storage completion makes A Ready while B is still Running. If the policy preempts B, A may run immediately after dispatch; otherwise A waits until B yields or finishes.
The key test is: *could this task continue if a CPU were offered now?* A ready task can; a blocked task cannot until its event occurs. "Not currently executing" alone does not distinguish them.

3.6 Process table versus a single PCB

The **process table** is an OS-wide collection or indexing arrangement for process-management records. A **PCB** is the record associated with one process. Saying that a table contains PCBs does not require one fixed array implementation: systems can index records using lists, maps or other structures.

When a timer interrupts a running task, the OS must preserve enough state to continue the exact instruction sequence later. It cannot reconstruct that point merely from the executable filename. The program counter records where execution resumes, registers preserve intermediate values, and the saved stack context preserves calls and local state. Scheduling fields and queue links determine which task is eligible next.

3.7 Suspension: distinguish two independent questions

Supporting seven-state models add **Ready Suspended** and **Blocked Suspended**. Ask both: "is the task able to proceed?" and "is the required process image resident under this swapping model?"

State	Event condition	Residency in the traditional model
Ready	Can proceed	Resident and awaiting CPU
Blocked	Required event not complete	Resident but unable to proceed
Ready Suspended	Can proceed	Removed from active residency; must be brought back before dispatch
Blocked Suspended	Required event not complete	Removed from active residency and still awaiting the event

If the awaited event completes while a task is Blocked Suspended, it becomes Ready Suspended, not Running. Bringing a still-blocked task back gives Blocked; bringing a ready-suspended task back gives Ready. These models explain swapping/suspension; they are not a claim that every modern OS uses exactly these named states.

4 Threads and multithreading

4.1 What is shared and what is private?

A **thread** is a unit of execution within a process. Each thread has a thread ID, program counter, register set and stack. Threads in the same process normally share the code, address space, global data, heap and process resources such as open files.

Private to each thread	Usually shared within the process
Program counter and CPU registers	Program code and address-space mappings
Stack and thread-local execution state	Global/static data and heap objects
Thread identifier and scheduling state	Open-file resources and other process-wide resources

A shared address space makes communication easy but makes unintended interference possible. Two threads that increment the same variable need appropriate synchronization. Different stacks do not protect shared heap data.

4.2 Benefits and limits

Responsiveness: a user-interface thread can remain active while another handles lengthy work. **Resource sharing:** related activities naturally access common data. **Economy:** creating/switching threads often costs less than doing so for separate processes, because some resources are shared. **Scalability:** runnable threads can exploit multiple cores.

These are opportunities, not automatic speed guarantees. Synchronization overhead, sequential work and excessive contention can limit gains. A failure in one thread can affect the entire process because the address space is shared.

4.3 User and kernel threads; mapping models

User-level threads are managed by a library/runtime. **Kernel-level threads** are recognised and scheduled by the OS. A user-level abstraction may map to one or several kernel threads.

Model	Mapping	Main consequence
Many-to-one	Many user threads to one kernel thread	No simultaneous execution of those threads on multiple cores. A blocking call that blocks the sole kernel thread can stall the group, unless the runtime avoids that blocking arrangement.
One-to-one	Each user thread maps to a kernel thread	Independent kernel scheduling and multicore parallelism; more kernel resources are needed.
Many-to-many	Many user threads multiplexed over several kernel threads	Combines runtime scheduling with a pool of kernel execution contexts; more complex coordination.
Two-level variant	Many-to-many with some bound mappings	Allows particular user threads to be bound to kernel threads. Supporting terminology.

Do not infer the mapping from the word "thread" alone. In modern systems, runtimes can provide lightweight tasks over kernel threads; the syllabus concepts describe the underlying relationships.

Worked example: Concurrency without parallelism

Three user threads mapped to one kernel thread can take turns and remain logically concurrent. Even on an eight-core machine they cannot all run simultaneously through that single kernel execution context. With three independently scheduled kernel threads and enough available cores, parallel execution becomes possible.

Further reading: OSC 9e, Chapter 4; OSTEP Chapter 26 for shared-state motivation.

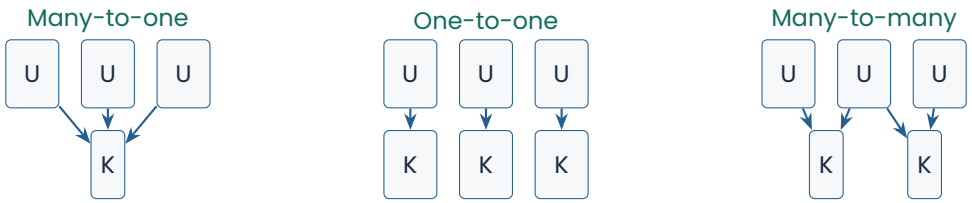
4.4 Choose processes or threads deliberately

Separate processes are useful when activities need stronger fault/isolation boundaries or independently managed execution environments. Threads are useful when related activities should share a working address space and communicate frequently. A process can contain multiple threads, so these choices can be combined.

Worked example: A server handling requests

A server can use a thread to handle one client's work while another thread remains available for incoming requests. Threads share the server's cache, so cached data need not be transferred through IPC. They must nevertheless protect shared updates. A separate worker process isolates its private memory more strongly, but access to a common cache requires an explicit sharing/communication arrangement.

4.5 Visualise the mapping models



U denotes a user-managed thread and K a kernel scheduling context. Arrows represent the mapping/multiplexing relationship, not permanent hardware wiring. Parallel execution requires several runnable kernel contexts and available cores. The thread library can schedule users within its contexts; the OS schedules the contexts it recognises.

5 Scheduling in the process life cycle

This section explains the Module 1 role of scheduling. Module 2 develops the CPU algorithms and calculations.

5.1 Queues and schedulers

A **ready queue** represents processes/threads eligible to use the CPU. **Device or event queues** hold tasks waiting for particular operations or conditions. Queues need not be literal FIFO lists; a scheduling policy can use a priority structure.

Scheduler	Decision	Typical role
Long-term/job	Which jobs to admit into the active workload	Controls degree of multiprogramming and workload mix. Some interactive systems have no separate long-term admission scheduler.
Short-term/CPU	Which ready task runs next	Runs frequently and must make decisions quickly.
Medium-term	Which tasks to suspend and later resume	Adjusts active memory/workload pressure in systems that use suspension/swapping.

A **CPU-bound** process spends relatively more time computing, often with longer CPU bursts. An **I/O-bound** process spends relatively more time waiting for I/O and often has shorter CPU bursts. A suitable mix can improve resource utilisation.

The **dispatcher** implements the selected CPU assignment: context switch, appropriate mode transition and transfer to the selected execution point. The scheduler chooses; the dispatcher hands over control. **Dispatch latency** is the time needed for that handover.

Worked example: A queue transition

An editor is running when it requests data from a device and must wait. It leaves the CPU and enters a waiting queue. The short-term scheduler chooses a ready computation process. When the device reports completion, the editor becomes ready again; it may still wait for CPU service.

6 Process creation and termination

6.1 Creation: identity, resources and execution

A running process can create another process, giving a **parent–child** relationship. Such relationships form process trees, though exact lifecycle rules are OS-specific.

1. Allocate an identifier and process-management structures.
2. Establish an address space and execution context.
3. Assign/inherit appropriate resources and access rights.
4. Load a new program or duplicate an existing execution environment, depending on the creation interface.
5. Make the new process eligible for scheduling.

A child may share some parent resources, inherit references to them or receive separate resources. Parent and child may execute concurrently; a parent may also wait for a child. A copied address space is not the same as a shared writable address space.

6.2 Unix-style fork, exec, wait and exit

These API names are useful illustrations; they are not the only possible design.

Operation	Meaning
`fork()`	Creates a child. On success, returns 0 in the child and the child's PID in the parent; on failure returns -1 in the parent and creates no child.
`exec` family	Replaces the calling process's program image with a specified program. A successful exec does not create an additional process and does not return to the old program.
`wait/waitpid`	Lets a parent obtain termination information and reap a child; a blocking form may wait until a relevant child terminates.
`exit`	Ends the calling process normally and supplies termination status.

After a successful fork, both parent and child continue from the call's return point with different return values. Their writable memory is logically separate. **Copy-on-write** can postpone copying physical pages until one process writes; it is an optimisation, not a change to the logical separation. Inherited file descriptors may refer to shared underlying open-file descriptions, so "everything is independent" is also inaccurate.

```
pid = fork()
if pid < 0:
    report_creation_failure()
else if pid == 0:
    exec(new_program, arguments)
    // Reached only if exec failed.
    report_exec_failure()
    terminate_child_with_error()
else:
    wait_for_child(pid)
    continue_parent_work()
```

Without an explicit ordering mechanism, you cannot predict which process executes its next instruction first. A scheduler may run either one.

6.3 Termination and cleanup

A process may terminate normally, encounter a fatal error or be terminated through an OS-authorised operation. The OS reclaims resources as appropriate, records status and wakes interested waiters. Closing a reference to a shared resource does not necessarily destroy the resource while others still use it.

A **zombie** in Unix-like terminology is a terminated child whose status has not yet been collected; it is not still executing and does not retain an ordinary running address space. An **orphan** is a live child whose parent has terminated; it can be adopted by an appropriate reaper or designated parent. An orphan is not automatically a zombie. Some systems or policies terminate descendants when a parent exits (cascading termination), but this is not a universal rule.

Worked example: Counting processes after repeated fork calls

Suppose one initial process executes three successive successful `fork()` calls, every child reaches the remaining calls, and no process exits early. The population doubles after each call: $1 \to 2 \to 4 \to 8$. There are **8 total processes and 7 newly created children**. In general the total is 2^n under these assumptions. Conditional calls, failures and early exits change the answer.

Three frequent mistakes

`exec` changes the program image rather than creating a second process. The child receives 0 from `fork`, not its own PID as the return value. A parent waiting for termination is waiting for an event, not competing in the ready queue during that wait.

Further reading: OSC 9e, Chapter 3; OSTEP Chapter 5.

6.4 Track the address spaces after creation

Worked example: A copied variable is not an IPC channel

Before fork, a process has a private variable `x=10`. After a successful fork, each process has the corresponding logical value 10. The child changes its private x to 20; the parent's x remains 10. Copy-on-write may initially share physical storage, but a write preserves the separate-process semantics.
To communicate the child's new value, use a suitable IPC mechanism or an explicitly shared mapping. A parent waiting for termination obtains completion status; wait does not generally transfer every changed child variable back to the parent.

Creation and program replacement answer different questions. Fork answers "create another process?" Exec answers "which program should this process now execute?" A shell can combine them: create a child, replace that child's image with the requested command, and wait or continue according to foreground/background behaviour.

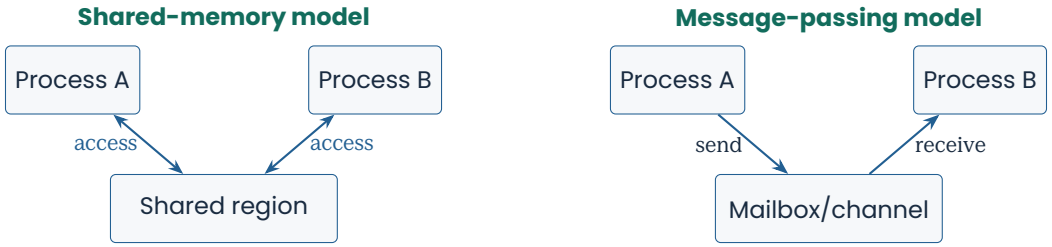
The parent–child relationship does not give the parent unlimited ability to terminate arbitrary other processes. Permissions and OS rules govern termination requests. Completion status also has a lifecycle: execution can end before the record needed for a parent's collection has been removed.

7 Inter-process communication

7.1 Why processes cooperate

Processes cooperate to share information, divide computation, organise a service or improve responsiveness. Because separate processes normally have protected address spaces, communication requires an OS-supported arrangement. Two central models are **shared memory** and **message passing**.

Communication answers "how is information transferred?" Synchronization answers "when may each participant proceed?" The two concerns are related but distinct. Merely making a memory region accessible does not ensure correctly ordered access.



7.2 Shared-memory systems

The OS creates or exposes a region that multiple processes can map into their address spaces. After setup, participants can access it with memory operations, subject to the arrangement's permissions and coherence rules.

1. Create/open the shared-memory object or region and arrange access rights.
2. Attach/map it into the participating processes' address spaces.
3. Agree on a data layout and a protocol for readers and writers.
4. Coordinate access using locks, semaphores or another correct synchronization method.
5. Unmap/close and remove resources when the design requires it.

The mapped virtual addresses need not be identical in different processes; offsets or other suitable representations can avoid assuming a common pointer value. Shared data must remain well-formed when several participants access it.

Advantages: efficient access to large amounts of data and flexibility in data structures; repeated transfers can avoid system calls and copies after setup. **Costs:** participants must handle synchronization and ownership carefully, and shared-memory access requires a suitable shared addressable environment. Setup, contention, copying elsewhere in the application and cache effects determine actual performance; shared memory is not always the fastest choice.

Worked example: Shared-memory producer and consumer

A producer stores records in a circular buffer; a consumer reads them. The participants must agree on where the next record goes and whether an item or free slot exists. If they concurrently update a shared count without protection, the count may be wrong even though the memory mapping works perfectly. The complete bounded-buffer solution is in Module 2.

7.3 Message-passing systems

Participants exchange messages through operations such as `send(destination, message)` and `receive(source, message)`. The mechanism transfers data and may also provide synchronization. It is useful where processes do not share ordinary memory, including distributed communication.

Direct communication names the other participant explicitly. A direct link may be symmetric (both name each other) or asymmetric (a receiver accepts from a source without naming it beforehand). **Indirect communication** uses a mailbox/port/channel: a sender deposits a message and a receiver obtains it. Multiple senders or receivers require defined delivery/selection rules. Mailbox names and handles depend on the interface; there is no universal requirement that they be integer IDs.

Blocking and nonblocking operations

A **blocking send** waits until the interface's completion condition is met; that condition may be acceptance into a buffer or receipt by the peer. A **nonblocking send** returns without waiting for that condition, subject to its defined success/error rules. A **blocking receive** waits for a suitable message. A **nonblocking receive** returns promptly with a message or an indication that none is available.

When both operations require the sender and receiver to meet for a transfer, the exchange is a **rendezvous**. Do not infer that every blocking send necessarily waits until the receiver has processed the message; completion semantics are interface-specific.

Buffering capacity

Capacity model	Consequence
Zero capacity	No pending-message queue; a transfer requires coordination with a receiver, commonly a rendezvous.
Bounded capacity	At most N pending messages; a blocking sender waits for space when full. A nonblocking interface may instead report failure/full status.
Theoretically unbounded	Capacity never becomes the reason to wait in the abstract model. Real systems have finite resources, and other completion requirements can still cause waiting.

Pipes are a familiar local communication mechanism; ordinary pipes are commonly byte streams rather than a general message-boundary-preserving mailbox. Named channels, queues and network endpoints provide other designs. These are examples supporting the two main syllabus models.

7.4 Shared memory versus message passing

Aspect	Shared memory	Message passing
Data exchange	Access a mutually available region	Transfer through send/receive operations
Setup	Establish mappings and permissions	Establish endpoints/link/mailbox
Coordination	Explicit shared-data protocol needed	Some ordering/blocking may be built into the interface; application protocol still matters
Performance	Can avoid repeated kernel-mediated copies after setup	Can incur transfer/copy overhead; actual implementations vary
Isolation	Shared writes can corrupt common state	Address spaces can remain separate; validate received messages
Distributed use	Requires a suitable shared-memory arrangement	Natural abstraction for networked participants

A complete 9-mark IPC answer

Define IPC and both models, draw two processes with either a shared region or a channel, explain access/transfer and synchronization, then compare them. For message passing, include direct/indirect naming, blocking/nonblocking operations and the three buffering capacities.

Further reading: OSC 9e, Chapter 3 IPC; UIC process notes.

7.5 Separate data transfer from the communication protocol

A communication mechanism does not by itself tell the receiver whether a whole record is ready, who owns a buffer or whether a reply is required. Participants must agree on a **protocol**: the meaning of the data and the permitted sequence of operations.

Worked example: Sending a measurement

With shared memory, A writes the measurement into a protected slot and then publishes availability using the chosen synchronization mechanism. B waits for availability, reads a complete measurement and returns the slot. Reading before publication may expose unfinished data.

With messages, A sends an agreed record to a mailbox and B receives it. B may send an acknowledgement if the application requires confirmation that the measurement was *processed*. Successful queue insertion alone does not establish that B has processed it.

7.6 Understand the two-index circular buffer

An introductory ring buffer sometimes uses only insertion and removal indices. It treats `in == out` as empty and `(in+1) mod N == out` as full. One slot must be left unused so the two states do not have identical indices: capacity is $N - 1$ in that design.

This is a representation choice, not a physical law. A protected count or the complete empty/full semaphore protocol can distinguish all N occupied slots from zero occupied slots. With $N = 4$,

the two-index empty-slot design stores at most 3 items; the properly synchronised count/permit design can store 4.

7.7 Which mechanism fits which communication?

Mechanism	Useful distinction
Shared memory	Repeated access to common data; explicit consistency and lifetime rules are essential.
Pipe	Often a local byte stream. A reader may need to assemble records from bytes; a single read need not match one complete application record.
Message queue/mailbox	Exchanges records through a defined queue with capacity and delivery rules. Naming differs between APIs.
Socket/channel	Provides an endpoint abstraction for local or network communication; stream and message-oriented interfaces have different semantics.

These examples support the syllabus's two main models. Choose from data volume, locality, isolation, required ordering and the available interfaces rather than declaring one mechanism universally best.

II Module 2: Scheduling and synchronization

8 CPU scheduling: basic concepts and criteria

8.1 CPU–I/O burst cycle

Execution often alternates between a **CPU burst**, during which instructions execute, and an **I/O burst/wait**, during which a process cannot proceed until an operation completes. CPU scheduling selects a task from the ready population whenever CPU assignment is required. Systems may schedule kernel threads rather than whole multithreaded processes; textbook numerical problems usually call each schedulable task a process.

A CPU-bound workload tends to have relatively long CPU bursts; an I/O-bound workload tends to have shorter CPU bursts separated by waits. Scheduling influences both utilisation and responsiveness. The dispatcher performs the handover described in Module 1.

8.2 Preemptive and nonpreemptive scheduling

Nonpreemptive scheduling: once selected, a process keeps the CPU until it terminates or voluntarily blocks/relinquishes it. **Preemptive scheduling:** the OS can take the CPU away from a runnable process, for example at a timer interrupt or when a higher-priority task becomes ready.

Scheduling decisions can arise when a process:

1. changes from Running to Waiting;
2. changes from Running to Ready;
3. changes from Waiting to Ready;
4. terminates.

Cases 1 and 4 require a replacement if work is ready. Cases 2 and 3 are associated with possible preemption; a nonpreemptive policy does not displace the current runnable process simply because a new one becomes ready. A timer tick permits an OS decision but does not imply every policy must switch tasks on every tick.

Preemption improves control over response time but adds switching overhead and makes synchronization important: a thread may be interrupted while modifying shared state. Synchronization is also necessary on multiple cores even without time-slice preemption.

8.3 Scheduling criteria

Criterion	Definition	Desired direction
CPU utilisation	Fraction of the chosen observation interval during which the CPU is busy	Maximise
Throughput	Completed processes/jobs per unit time	Maximise
Turnaround time	Elapsed time from arrival/submission to completion	Minimise
Waiting time	Total time spent in the ready queue	Minimise
Response time	Time from arrival until the first CPU service in these numerical problems	Minimise
Fairness	Avoid unreasonable exclusion; provide appropriate shares	Policy-dependent
Deadline satisfaction	Meet timing requirements when the workload has deadlines	Meet constraints

In an interactive application, "response" may refer to the first externally visible response rather than literally the first instruction. Use the first-start definition for the standard CPU scheduling tables in this guide. No one policy optimises every criterion for every workload.

8.4 The formulas and their assumptions

Single-CPU, single-burst numerical problems

Let AT = arrival time, BT = total CPU burst, CT = completion time and First = first execution time.

$$\text{TAT} = \text{CT} - \text{AT}, \qquad \text{WT} = \text{TAT} - \text{BT}, \qquad \text{RT} = \text{First} - \text{AT}.$$

The waiting formula assumes there is no additional I/O/blocked interval and the usual zero-overhead model. With I/O, turnaround also includes blocked time; subtracting only CPU service would not isolate ready-queue waiting. Compute an average by summing the individual values and dividing by the number of processes.

$$\text{CPU utilisation} = \frac{\text{busy CPU time}}{\text{observation interval}} \times 100\%, \quad \text{throughput} = \frac{\text{number completed}}{\text{observation interval}}.$$

State the observation interval. Starting at time zero gives a different utilisation from starting only when the first job arrives if the CPU was initially idle.

8.5 A reliable numerical method

1. Write the arrivals, bursts, priorities, quantum and all tie rules.
2. Track the **ready** set/queue at each decision; never choose a process before it arrives.
3. Draw a timeline with both the process name and every relevant time boundary. Include idle intervals.
4. Find each first-start time and the end of its *last* interval, CT.
5. Compute TAT, WT and RT, then their averages.

6. Check that each process received exactly BT units, all values are nonnegative and the timeline accounts for busy and idle time.

For preemptive policies, an intermediate stop is not completion. Waiting time includes *all* ready intervals; response counts only the delay before the first start.

8.6 Read a CPU burst as service, not elapsed lifetime

A burst of 4 units means the task needs 4 units of CPU service. If it executes for 1, waits ready for 5, then executes for 3, it has received its burst even though 9 units passed between its first start and completion. Preemption divides service into intervals; it does not increase the requested useful service in the zero-overhead model.

A task's turnaround also includes delay before its first service. Thus a low response time does not imply low turnaround: a job can begin promptly and finish much later. CPU utilisation can be 100% while a particular job suffers a long wait.

8.7 The common event loop behind the policies

```
while unfinished jobs exist:
    add all newly arrived jobs to the ready structure
    if no job is ready and none is running:
        advance to the next arrival; record an idle interval
    else:
        choose or retain a job according to the policy
        execute until completion or the next relevant event
        update remaining service and the ready structure
```

"Relevant event" depends on policy. A new arrival triggers reconsideration for SRTF/priority preemption, but does not interrupt a nonpreemptive burst. A quantum expiry triggers RR requeueing. The ready structure can be a FIFO queue or a policy-specific ordered structure; it must not contain jobs that have not arrived.

9 Common data set for the scheduling algorithms

The following deliberately includes different arrival times, unequal bursts and priority ties. All examples use one CPU, one CPU burst per process and **zero context-switch overhead**. Smaller priority numbers mean higher priority.

Process	AT	BT	Priority
P_1	0	7	3
P_2	2	4	1
P_3	3	1	4
P_4	5	4	2
P_5	6	2	1

Tie rules: earlier arrival, then smaller process number. For SRTF and preemptive priority, preempt only for a *strictly* better remaining time/priority; retain the running task on equality. For RR, arrivals at a quantum boundary enter the queue *before* the just-expired process is requeued. A question can specify a different convention; then show and follow it consistently.

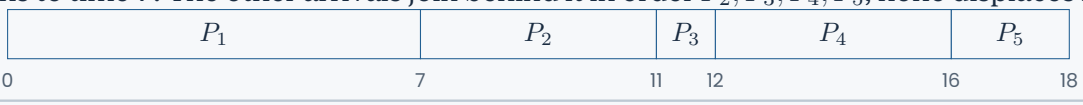
10 First Come First Served (FCFS)

FCFS serves ready processes in arrival order. The standard CPU policy is **nonpreemptive**; its ready queue is FIFO. When a process blocks or terminates, the oldest waiting ready process is selected.

Strengths: simple to implement; no repeated time-slice preemptions; each job has a clear place in the queue. **Weaknesses:** a long burst can hold up many short bursts, giving poor average waiting and response times. In the **convoy effect**, shorter or I/O-heavy tasks wait behind a long CPU-bound task, which can also reduce effective device utilisation as the workload moves between CPU and I/O activity.

Worked example: FCFS with staggered arrivals

P_1 runs to time 7. The other arrivals join behind it in order P_2, P_3, P_4, P_5; none displaces it.



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	7	0	7	7	0	0
P_2	2	4	7	11	9	5	5
P_3	3	1	11	12	9	8	8
P_4	5	4	12	16	11	7	7
P_5	6	2	16	18	12	10	10

Average turnaround $= 9.60$, waiting $= 6.00$ and response $= 6.00$. For example, P_4 completes at 16: TAT $= 16 - 5 = 11$, WT $= 11 - 4 = 7$, RT $= 12 - 5 = 7$. For any single-burst nonpreemptive schedule here, RT equals WT because there is just one continuous execution interval per process.

FCFS is about the ready queue

If a real process returns from I/O, it re-enters the ready queue according to policy. Do not interpret FCFS as giving the same process permanent priority because it was created first. The one-burst numerical model avoids these extra queue visits.

10.1 Build FCFS by maintaining the queue

Append newly ready jobs at the back. When the CPU becomes free, remove the front job and run its whole next burst. For arrivals at 0, 2 and 3 while the first burst lasts to 7, the later jobs accumulate; the CPU does not wait intentionally for a future shorter job.

If every job arrives at 0, each waiting time is the sum of the bursts before it. With bursts 8, 2, 1 in that order, waits are $0, 8, 10$, averaging 6. Reordering them as 1, 2, 8 gives waits $0, 1, 3$, averaging $4/3$. FCFS therefore depends strongly on arrival/order, even when the amount of total CPU work is unchanged.

FCFS suits simple batch admission where turn order is the primary rule. It does not supply short interactive responses merely because it avoids quantum preemption. Once admitted into a finite FIFO queue with finite service, a job eventually reaches the front; the convoy effect is long waiting, not automatically starvation.

11 Shortest Job First (SJF) and its preemptive variant

11.1 Nonpreemptive SJF

At each selection, choose the ready process with the shortest next CPU burst. Once it starts, it keeps the CPU until the burst ends. SJF uses the *next CPU burst*, not necessarily the total duration of the whole program.

For a fixed collection of simultaneously available jobs with known burst lengths, ordering them by increasing burst length minimises total/average waiting time. A short job placed before a long one reduces how much service the following job must wait for. This result does not mean nonpreemptive SJF is clairvoyantly optimal for every staggered arrival pattern.

Actual next bursts are often unknown. A common estimate is exponential averaging:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n, \qquad 0 \le \alpha \le 1,$$

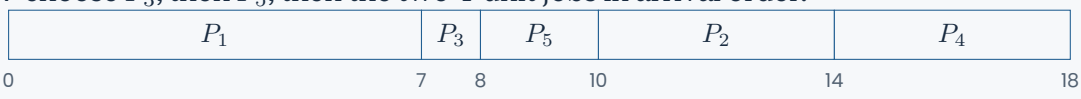
where t_n is the measured last burst and τ_n is its previous prediction. Larger α gives more weight to recent behaviour. With $\alpha = 0$, new measurements are ignored; with $\alpha = 1$, the most recent burst alone is used.

Worked example: Predicting the next burst

If the previous prediction is 6 units, the actual last burst is 4 units and $\alpha = 0.5$, the new prediction is $0.5(4) + 0.5(6) = 5$ units. An estimate is not advance knowledge of the future burst.

Worked example: Nonpreemptive SJF

At time 0 only P_1 is ready, so it runs for all 7 units even though shorter jobs arrive later. At time 7 choose P_3, then P_5, then the two 4-unit jobs in arrival order.



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	7	0	7	7	0	0
P_2	2	4	10	14	12	8	8
P_3	3	1	7	8	5	4	4
P_4	5	4	14	18	13	9	9
P_5	6	2	8	10	4	2	2

Average turnaround $= 8.20$, waiting $= 4.60$ and response $= 4.60$.

Limitation: starvation. A long ready burst may be postponed repeatedly if shorter ones keep arriving. Fairness measures such as ageing can mitigate postponement in a suitable combined policy.

11.2 Shortest Remaining Time First (SRTF)

SRTF is the preemptive form commonly discussed with SJF. A newly ready process can displace the current one if its burst is shorter than the current process's *remaining* service. The syllabus names SJF; understanding this variant prevents ambiguity when an exam specifies preemptive SJF.

Worked example: Recompute remaining time at each arrival

At time 2, P_1 has 5 units left, so P_2 with 4 units preempts it. At time 3, P_2 has 3 left; P_3 needs only 1 and runs to time 4. P_2 then resumes. At time 6, P_2 has 1 left, so the newly arrived P_5 with 2 does *not* preempt it.



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	7	0	18	18	11	0
P_2	2	4	2	7	5	1	0
P_3	3	1	3	4	1	0	0
P_4	5	4	9	13	8	4	4
P_5	6	2	7	9	3	1	1

Average turnaround $= 7.00$, waiting $= 3.40$ and response $= 1.00$. P_1 waits while it is interrupted from time 2 to 13: WT $= 18-0-7 = 11$, but RT $= 0$. Completion time must come from its final interval, 13–18.

SJF versus SRTF

SJF compares full next bursts only when selecting a new task. SRTF compares *remaining* bursts and can interrupt a running one. State the version before drawing a Gantt chart.

11.3 Why short-before-long reduces average waiting

Suppose two simultaneously ready jobs have bursts $a > b$ and the same delay w before this pair begins. Running the long one first gives combined waiting $w + (w + a) = 2w + a$. Running the short one first gives $2w + b$. The second order saves $a - b$ waiting units, without changing when the pair finishes or the later jobs' start times.

Repeatedly exchanging inverted neighbouring pairs gives increasing-burst order. This proves the familiar optimality result for the stated all-ready, known-burst, nonpreemptive model. For staggered arrivals, selecting a ready long task before an unseen future short task is not a violation of the rule; the online scheduler does not know or deliberately wait for every future arrival.

11.4 Distinguish estimation from selection

An estimate predicts the next burst. SJF selects using those estimates; SRTF uses an appropriate estimate of remaining service. Good predictions help, but bad predictions can produce a schedule unlike the ideal known-burst example. Exponential averaging balances older and newer measurements; it cannot guarantee a future burst's exact length.

At each SRTF arrival, subtract service already received from the current job before comparing. A task that started with 7 units and has run 5 has only 2 left. A new 3-unit job is shorter than the *original* burst but should not displace that 2-unit remainder under SRTF.

12 Priority scheduling

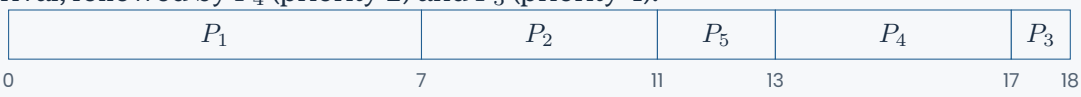
Choose the ready process with the highest priority. Priorities can be based on internal factors such as resource use or external factors such as an assigned importance. A smaller number can mean higher priority, or the opposite; the question must establish the convention.

Nonpreemptive priority leaves the running process until it blocks/finishes. **Preemptive priority** can displace it when a strictly higher-priority task becomes ready. Equal priorities require a tie policy, commonly FCFS.

12.1 Nonpreemptive worked example

Worked example: Priority selects among ready jobs

P_1 is the only process at time 0 and finishes at 7. The priority-1 jobs P_2 and P_5 go first, ordered by arrival, followed by P_4 (priority 2) and P_3 (priority 4).



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	7	0	7	7	0	0
P_2	2	4	7	11	9	5	5
P_3	3	1	17	18	15	14	14
P_4	5	4	13	17	12	8	8
P_5	6	2	11	13	7	5	5

Average turnaround $= 10.00$, waiting $= 6.40$ and response $= 6.40$.

12.2 Preemptive worked example

Worked example: Higher priority arrives

At time 2, P_2 (priority 1) displaces P_1 (priority 3). No subsequent arrival outranks P_2, so it finishes at 6. Then P_5 runs, followed by P_4, the rest of P_1 and finally P_3.



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	7	0	17	17	10	0
P_2	2	4	2	6	4	0	0
P_3	3	1	17	18	15	14	14
P_4	5	4	8	12	7	3	3
P_5	6	2	6	8	2	0	0

Average turnaround $= 9.00$, waiting $= 5.40$ and response $= 3.40$.

12.3 Starvation, ageing and supporting terminology

Starvation/indefinite postponement occurs when a low-priority process keeps waiting while higher-priority work is selected. **Ageing** gradually improves the priority of a waiting process, so prolonged waiting eventually affects selection. If smaller values mean higher priority, an ageing rule might decrease a priority value over time, subject to limits.

Priority inversion is different: a high-priority task waits for a lock held by a lower-priority one; a medium-priority task can delay the holder. **Priority inheritance** can temporarily raise the lock

holder's scheduling priority. This is supporting detail connecting priority scheduling and locks; it is not the same as ageing.

12.4 Priority changes must have a defined meaning

Priority is a preference assigned by policy, not a measure of burst length. A short job can have low priority, and an important job can need a long burst. Always say whether the numerical priority increases or decreases with importance.

Worked example: An explicit ageing rule

Suppose smaller values are higher priority and a waiting job begins at 6. A policy improves it by one level after each 5 units of ready waiting, stopping at priority 1. After 15 units it is at 3; after 25 it is at 1. To avoid indefinite postponement at the top level, equal-priority selection must also be appropriate, for example FIFO. The arithmetic describes this chosen policy, not a universal ageing formula.

12.5 A priority-inversion timeline

Low-priority L holds a lock. High-priority H becomes ready, attempts that lock and blocks. Medium-priority M can now execute instead of L; H is indirectly delayed by M, even though H outranks it. Priority inheritance temporarily lets L inherit the required higher priority so it can finish the protected work and release the lock. This solves a lock-owner delay mechanism; ageing solves long ready-queue postponement.

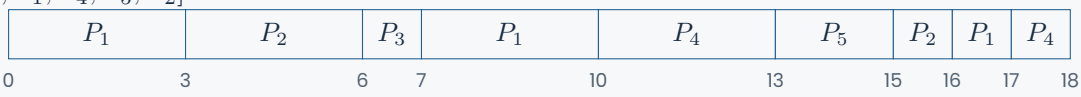
13 Round Robin (RR)

RR keeps a FIFO ready queue and assigns a maximum **time quantum** q to each turn. A process that completes or blocks before using q relinquishes the CPU early. A still-runnable process whose quantum expires goes to the back of the queue. RR is preemptive and is widely used to explain time-sharing response.

13.1 Queue method: quantum 3

Worked example: RR with arrival-at-boundary convention

P_1 runs from 0 to 3. P_2 arrived at 2; P_3 arrives at 3 before P_1 is requeued. Thus the ready queue at 3 is $[P_2, P_3, P_1]$. At time 6, P_4 and P_5 have joined, and P_2 returns with 1 unit left: the queue is $[P_3, P_1, P_4, P_5, P_2]$.



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	7	0	17	17	10	0
P_2	2	4	3	16	14	10	1
P_3	3	1	6	7	4	3	3
P_4	5	4	10	18	13	9	5
P_5	6	2	13	15	9	7	7

Average turnaround $= 11.40$, waiting $= 7.80$ and response $= 3.20$. P_4 runs at 10–13 and 17–

18, so its first time is 10, CT is 18, TAT is 13, WT is 9 and RT is 5. A process uses less than a full quantum in its final turn if its remaining burst is shorter.

13.2 Choosing the quantum

If q is large enough that every next CPU burst finishes within one turn, RR behaves like FCFS for that workload. Small q can improve the time until each ready task gets a turn, but creates more switching overhead. A practical quantum balances responsiveness with cost.

For n continually ready tasks and no overhead or new arrivals, a task just requeued waits at most $(n-1)q$ until its next turn. This is a bound for that fixed setting, not a promise for every changing system. With one runnable task, a timer expiry need not cause a switch to a different task.

Worked example: Estimating switching overhead

If each quantum of useful execution lasts 4 ms and each handover takes 0.2 ms, the repeated full-quantum model gives an overhead fraction of $0.2/(4+0.2) \approx 4.76\%$. If the useful quantum is 1 ms, it becomes $0.2/1.2 \approx 16.67\%$. Actual fractions depend on early completion, blocking and whether a real handover occurs.

RR is not repeated shortest-job selection

Keep the FIFO queue and remaining bursts separately. New arrivals append in the stated order. Do not sort the queue by burst, and do not give an unfinished process another immediate turn while older ready tasks wait.

13.3 Write the queue beside the Gantt chart

For the main $q = 3$ example, the following queue is the state *after arrivals and completion/requeue handling, before dispatch*. The front entry runs next.

Time	Ready queue	Why it has this order
0	$[P_1]$	Only the first job has arrived.
3	$[P_2, P_3, P_1]$	Boundary arrival precedes requeued P_1.
6	$[P_3, P_1, P_4, P_5, P_2]$	P_4, P_5 join before the expired P_2.
7	$[P_1, P_4, P_5, P_2]$	P_3 completed after just 1 unit.
10	$[P_4, P_5, P_2, P_1]$	P_1 returns with 1 unit left.
13	$[P_5, P_2, P_1, P_4]$	P_4 returns with 1 unit left.
15	$[P_2, P_1, P_4]$	P_5 completed.
16	$[P_1, P_4]$	P_2 completed.
17	$[P_4]$	P_1 completed.
18	$[\,]$	All work completed.

This technique prevents accidentally sorting by burst or requeueing a finished task. The quantum is a *maximum*, so unused time in a finishing turn is not forced idle time.

Worked example: Compute waiting directly from interrupted intervals

P_4 arrives at 5, first runs at 10, stops at 13, and resumes at 17. Its ready waits are 5–10 and 13–17: $5+4=9$ units. Its response is only the first wait, 5. The formula check gives $18-5-4=9$, agreeing with the timeline.

14 Comparison and numerical cross-checks

Policy	Selection rule	Main concern
FCFS	Oldest ready arrival	Long-job convoy; weak short-job response
SJF	Shortest next ready burst	Burst prediction; long-job starvation
SRTF	Shortest remaining ready service	Preemption overhead; long-job postponement
Priority	Highest ready priority	Low-priority starvation; tie convention
RR	FIFO turns up to q	Quantum versus switching overhead

Common example	Average TAT	Average WT	Average RT
FCFS	9.60	6.00	6.00
SJF	8.20	4.60	4.60
SRTF	7.00	3.40	1.00
Priority, nonpreemptive	10.00	6.40	6.40
Priority, preemptive	9.00	5.40	3.40
RR, $q=3$	11.40	7.80	3.20

These values compare this particular workload and stated assumptions. They do not show that an algorithm universally dominates another. RR aims at opportunities to respond, not at minimising this example's average waiting time.

Worked example: Idle time and the observation interval

Let P_1 arrive at 3 with burst 2 and P_2 arrive at 6 with burst 1. The timeline is idle 0–3, P_1 3–5, idle 5–6, P_2 6–7. Observing 0–7, busy time is 3, utilisation is $3/7\times100\%\approx42.86\%$ and throughput is $2/7$ processes per unit time. Both have zero waiting and response time. Omitting idle gaps would falsely imply an arrival before time 3 or an impossible completion.

Further reading: OSC 9e, Chapter 6; OSTEP Chapter 7; UIC CPU Scheduling notes.

15 Race conditions and the critical-section problem

15.1 Why interleaving can change a result

A **race condition** occurs when the correctness or result of an operation depends on the timing/interleaving of concurrent accesses to shared state. The problem is uncontrolled access, not simply the presence of several processes.

An expression such as `count++` may require a read, an arithmetic operation and a write. These separate actions can interleave with another participant's actions. Multiple CPU cores can also execute conflicting actions simultaneously.

Worked example: Producer increments while consumer decrements

Initially `count = 5`. Both operations should together leave it at 5. One legal interleaving of unprotected elementary actions is:

Step	Producer	Consumer	Shared count
1	Read count into $r_P = 5$		5
2	Compute $r_P = 6$		5
3		Read count into $r_C = 5$	5
4		Compute $r_C = 4$	5
5		Store r_C	4
6	Store r_P		6

The producer's last store overwrites the decrement. Another interleaving ends at 4; a serial execution ends at 5. Enumerating all 20 order-preserving interleavings of these two three-action operations gives possible final values $\{4, 5, 6\}$. An indivisible correctly ordered operation or protection around the shared update prevents this lost-update pattern.

Race condition versus data race. "Race condition" is a broad algorithmic term. In languages such as C/C++, conflicting unsynchronised accesses to ordinary shared objects can constitute a *data race* with undefined behaviour; the simple read/store interleaving model is then not a reliable description of every possible compiled outcome. The teaching example explains why synchronization is needed.

15.2 Critical sections and three correctness requirements

A **critical section (CS)** is the part of a participant's program that accesses shared state/resources requiring coordination. A general structure is:

```
repeat:
    entry_section()       // Request permission to enter.
    critical_section()    // Access protected shared state.
    exit_section()        // Release permission.
    remainder_section()   // Other work.
```

The critical-section problem asks for a protocol satisfying:

1. **Mutual exclusion:** at most one participant executes the protected CS at a time.
2. **Progress:** if the CS is empty and some participants want to enter, selecting the next entrant cannot be postponed indefinitely by processes that are not participating in that decision. Remainder-section processes must not prevent progress.
3. **Bounded waiting:** after a participant requests entry, a bound exists on how many times others may enter before it is allowed to enter.

Bounded waiting bounds overtaking, not a fixed wall-clock delay. The reasoning assumes participants continue to receive execution opportunities and that critical sections finish; no protocol can make progress if an essential participant is permanently stopped while holding a resource.

15.3 Two failed software approaches

Strict alternation uses a shared `turn`. Process i waits until `turn == i`, enters, then sets `turn = j`. It can ensure mutual exclusion with suitable atomic access, but violates progress: if process 0 wants another entry while process 1 remains in its remainder section, process 0 may wait even though the CS is free.

Separate intent flags alone use `flag[i] = true` and then wait while the other's flag is true. If both set their flags before either checks, both can wait forever. Intent is necessary to represent interest, but without a conflict-resolution rule this approach does not ensure progress.

Identify which requirement fails

A protocol that prevents simultaneous entry can still be incorrect because it deadlocks, delays entry unnecessarily or permits starvation. State the particular failure and give an execution demonstrating it; "mutual exclusion holds" is only one part of a complete argument.

15.4 What must be protected?

Protect the *invariant*, not merely one convenient assignment. In a bank transfer, decreasing account A and increasing account B must be coordinated as a transfer; locking only an individual balance update may still let an observer see an incomplete pair. In a buffer, the stored item, index and availability state must follow one consistent protocol.

Independent processes with no relevant shared state need no mutual exclusion with each other for their private calculations. Cooperating participants need coordination when one can observe or modify state used by another. Read-only access may coexist safely; conflicting writes and read/write access require particular care.

Not all unpredictable ordering is a bug. Two independent tasks may print in either order if the application accepts both. A race is a correctness problem when the interleaving can violate the required result or invariant. Making one primitive atomic protects that primitive, not automatically every multi-step application operation.

16 Peterson's solution

16.1 Two-process algorithm and assumptions

For two participants P_0 and P_1, define shared `flag[2] = {false,false}` and a shared `turn`. `flag[i]` announces an interest in entry; `turn` resolves competition. For process i, let $j = 1 - i$:

```
repeat:
    flag[i] = true
    turn = j
    while flag[j] and turn == j:
        skip                // Busy wait.
    critical_section()
    flag[i] = false
    remainder_section()
```

The textbook proof assumes **atomic individual reads/writes and sequential consistency**: operations appear in a total order consistent with each participant's program order. The compound condition can involve separate reads; it need not be one indivisible multi-variable test. There are exactly two participants, and each sets/clears its own flag as shown.

16.2 Walk through simultaneous interest

Step	P_0 action	P_1 action	Consequence
1	`flag[0]=true`		P_0 announces interest.
2		`flag[1]=true`	Both are interested.
3	`turn=1`		P_0 yields to P_1.
4		`turn=0`	Last turn write yields to P_0.
5	Tests other flag true, turn not 1		P_0 enters.
6		Tests other flag true, turn 0	P_1 waits.
7	Leaves CS, clears flag[0]		P_1 can proceed.

If the turn writes occur in the opposite order, the other participant wins. Setting `turn=j` is a willingness to let the other go first, not a declaration of one's own immediate right to enter.

16.3 Why it satisfies the three requirements

Mutual exclusion. Consider one participant entering while the other's current request flag is true. It can pass only if turn favours itself. Now consider the ordering of the two turn assignments for overlapping requests. The *last* assignment favours the other participant, so its writer cannot pass while the other remains interested. If one participant passed before the other's flag was set, the later participant's own turn assignment favours the already-interested participant and makes the later entrant wait. In both cases, the later/losing participant cannot enter while the first stays in its CS. This chronological argument matters because values can change between separate loop reads.

Progress. If only P_i is interested, `flag[j]` is false and it can enter without waiting for P_j to act. If both are interested, the turn value favours one, so both cannot be blocked by the conflict simultaneously once the entry writes have occurred.

Bounded waiting. After P_i has set its flag and turn, P_j can get ahead at most once. On trying again, P_j writes `turn=i` and cannot repeatedly overtake the still-interested P_i. Finite CS duration and execution opportunities remain assumptions.

16.4 Limits and implementation caution

Peterson uses **busy waiting**, supports two participants in this form and depends on memory-operation ordering. Modern CPUs and compilers may reorder ordinary operations or expose weaker memory behaviour. In C/C++, plain shared variables are not a correct implementation, and `volatile` alone does not supply synchronization. Properly ordered atomics can express the teaching assumptions; production code should normally use established mutex/synchronization interfaces.

What the additional check establishes

The accompanying script explores 702 reachable states for two entries per participant, modelling separate atomic reads of the flag and turn under sequential consistency. It finds no state with both participants in the CS. This is a finite safety check; the reasoning above supplies the broader protocol explanation. It does not verify compiled code on a particular CPU or independently prove starvation freedom.

Further reading: OSC 9e, Chapter 5, Peterson's solution; UIC synchronization notes.

17 Synchronization hardware

17.1 Disabling interrupts: scope and limits

On a single processor, kernel code can sometimes protect a very short operation by disabling relevant interrupts, thereby preventing interrupt-driven preemption during it. This does not provide general mutual exclusion on a multiprocessor: another core can still access the data. Disabling interrupts also delays interrupt service and is privileged, so it is not a general user-program solution. Kernel designs combine appropriate locks and interrupt handling for the actual context.

17.2 Atomic test-and-set

Hardware provides indivisible read–modify–write operations. **Test-and-set** atomically returns the old value of a variable and sets it to true:

```
// Entire operation below is atomic in the abstraction.
TestAndSet(address):
    old = value_at(address)
    value_at(address) = true
    return old

shared lock = false
repeat:
    while TestAndSet(&lock) == true:
        skip
    critical_section()
    release_store(lock, false)
    remainder_section()
```

Only one contender can observe a false old value and acquire the lock. All later contenders find it true until release. A real lock needs the memory-order guarantees appropriate to acquisition and release, not merely atomic modification of the lock word.

Limitation: this simple spinlock ensures mutual exclusion under its assumptions but does not give bounded waiting. A newly arriving contender can repeatedly win over an older waiter. It also wastes CPU while waiting, especially if the owner is descheduled.

17.3 Atomic compare-and-swap (CAS)

CAS atomically compares a memory value with an expected value and writes a replacement only if they match. In this textbook interface it returns the old value regardless of success:

```
// Entire operation below is atomic.
CompareAndSwap(address, expected, replacement):
    old = value_at(address)
    if old == expected:
        value_at(address) = replacement
    return old

shared lock = 0
repeat:
    while CompareAndSwap(&lock, 0, 1) != 0:
        skip
    critical_section()
    release_store(lock, 0)
    remainder_section()
```

The winning contender finds 0 and changes it to 1. Some programming-language CAS APIs return a success boolean and modify an expected-value argument instead; do not confuse that interface with the old-value pseudocode above. CAS is a building block, not a guarantee that every algorithm built from it is fair or correct.

17.4 Adding bounded waiting: an explicit handoff protocol

Supporting detail from the textbook: atomic test-and-set alone does not impose an order, but a waiting array and cyclic handoff can do so. For n participants use shared `waiting[n]=false` and `lock=false`; process i runs:

```
repeat:
    waiting[i] = true
    key = true
    while waiting[i] and key:
        key = TestAndSet(&lock)
    waiting[i] = false
    critical_section()

    j = (i + 1) mod n
    while j != i and not waiting[j]:
        j = (j + 1) mod n
    if j == i:
        release_store(lock, false)
    else:
        waiting[j] = false       // Hand ownership to j.
    remainder_section()
```

On handoff the lock remains set, keeping new outsiders from acquiring it; the chosen waiter exits its loop through its cleared waiting flag. Cyclic selection prevents an announced waiter from being passed by more than $n - 1$ other entries under the textbook assumptions and normal progress assumptions. This is an abstract ordered-memory algorithm, not a drop-in implementation using ordinary C booleans.

Further reading: OSC 9e, Chapter 5 synchronization hardware; OSTEP Chapter 28.

17.5 Atomic exchange/swap: another hardware building block

An atomic exchange swaps a shared lock value with a private value. Initialise `lock=false`; each contender uses its own `key`:

```
repeat:
    key = true
    while key == true:
        AtomicSwap(lock, key)   // Swap both values indivisibly.
    critical_section()
    release_store(lock, false)
    remainder_section()
```

If the old lock is false, the swap sets lock true and gives the caller key false, so it enters. If the old lock is true, key stays true and the caller retries. The winning acquisition is indivisible; an ordinary three-assignment swap is not a substitute.

Exchange, TAS and CAS differ in their interfaces, but a simple retry loop built from any of them can still be unfair. Appropriate memory ordering must make protected data visible across release/acquire, not just prevent two lock-word winners.

18 Mutex locks

A **mutex** provides mutual exclusion with operations such as `lock/acquire` and `unlock/release`. A task acquires before the CS and releases after it. The acquisition must atomically decide ownership; an unprotected "if free then set busy" sequence can let two tasks enter.

```
mutex.lock()
try:
    update_shared_state()
finally:
    mutex.unlock()
```

The `try/finally` expresses that every successful acquisition must be paired with release even when the protected operation fails. It is not an additional synchronization primitive. A nonrecursive mutex is not generally safe to acquire twice by its current owner; recursive behaviour is a separate interface choice.

18.1 Spinning versus blocking

A **spinlock** repeatedly checks/attempts acquisition and stays runnable. It can be suitable for very short waits when the owner is executing on another core, and in contexts where sleeping is unavailable. A **blocking mutex** suspends a contending thread and wakes it when acquisition becomes possible; this avoids long CPU-consuming waits but introduces queueing and scheduling costs. Implementations can combine short spinning with blocking.

On a single CPU, a high-priority spinner can prevent a lower-priority lock holder from running unless scheduling/locking design addresses the situation. Choose based on execution context and expected wait, not a blanket rule that spinning is always fast or blocking is always preferable.

18.2 Ownership and correctness

A mutex has ownership semantics: normally the task that acquires it must release it. It protects the *same agreed shared state* only if every conflicting access follows the protocol. Multiple unrelated locks around the same variable do not ensure exclusion. A mutex is not automatically fair; liveness depends on its implementation and scheduling.

Keeping the CS short reduces contention. Blocking on another resource while holding a lock can create dependency cycles; the classic examples below show why acquisition order matters.

Further reading: OSC 9e, Chapter 5 mutex locks; OSTEP Chapter 28.

19 Semaphores

19.1 Atomic wait and signal

A **semaphore** is a synchronization object representing permits or events, manipulated through atomic operations commonly called `wait/P/down` and `signal/V/up`.

Permit-count convention used in the solutions

`wait(S)` obtains one permit: if $S > 0$, atomically decrement it and proceed; otherwise wait until a permit can be obtained. `signal(S)` makes one permit available, allowing a waiting participant to proceed as the implementation provides. Testing and taking the permit are coordinated atomically. In this convention the displayed available-permit count is nonnegative.

A **counting semaphore** can start at N to control access to N identical available units. A **binary semaphore** restricts the conceptual state to unavailable/available and can support exclusion or event signalling. Exact treatment of repeated signals depends on the interface; do not assume arbitrary over-signalling is valid.

19.2 Mutual exclusion and ordering are different uses

For exclusion, initialise a semaphore to 1:

```
wait(mutex_sem)
critical_section()
signal(mutex_sem)
```

For ordering, initialise `done = 0`. If action B must occur after A:

```
Process 1:                  Process 2:
    perform_A()                 wait(done)
    signal(done)                perform_B()
```

If process 2 arrives first, it waits. If process 1 signals first, a permit remains for process 2. The operation orders A before B; it does not necessarily serialize unrelated work. An initial value of 1 would incorrectly let B run before A.

Worked example: Three available devices

A pool has three equivalent devices. Initialise `available=3`; each user waits before obtaining a device and signals when returning it. Three users can proceed. A fourth waits until a return creates a permit. The semaphore limits how many are in use; additional protected bookkeeping may be needed to choose which specific device each user gets.

19.3 Blocking implementation and the negative-count convention

Textbooks may show another internal representation: decrement an integer on wait; if it becomes negative, enqueue and block the caller. On signal increment it; if it is still nonpositive, wake one queued waiter. In this representation, a negative value encodes waiting tasks rather than negative physical resources.

```
// Abstract internal operations, each coordinated atomically.
wait(S):
    S.value = S.value - 1
    if S.value < 0:
        add_current_task_to(S.queue)
        block_current_task()

signal(S):
    S.value = S.value + 1
    if S.value <= 0:
        task = remove_a_waiter_from(S.queue)
        wake(task)
```

The update, queue management and sleep/wakeup protocol must be coordinated so a wakeup is not lost between "decide to sleep" and "actually sleep". The code is an atomic abstract description, not ordinary assignments around arbitrary system calls. Short internal locks/atomic operations often protect the semaphore itself.

Do not mix the two count conventions in one calculation. In the available-permit convention, a waiter does not take a nonexistent token; in the internal signed convention, the decrement also records the wait.

19.4 Mutex versus semaphore

Aspect	Mutex	Semaphore
Main abstraction	Owned lock for exclusion	Permits or event/count coordination
Release responsibility	Normally the acquiring owner	Can be a different participant, as in producer signalling consumer
Initial state	Unlocked	Chosen count, e.g. 0, 1 or N
Concurrency allowed	One holder	Up to the permitted count for resource use
Typical use	Protect a shared structure	Resource pool, availability counts, ordered steps

A binary semaphore can implement an exclusion protocol, but that does not make all ownership, error-checking and scheduling semantics identical to a mutex.

19.5 Typical errors and liveness issues

Omitting a wait can violate exclusion; omitting a signal can keep later callers waiting forever; an extra signal can create permits that do not correspond to resources. Holding one semaphore while waiting for another can deadlock if the dependency order cycles. A waiter can starve if selection is unfair. Using semaphores does not by itself guarantee deadlock or starvation freedom.

Further reading: OSC 9e, Chapter 5; OSTEP Chapter 31.

19.6 Trace the two semaphore count conventions

For a semaphore initially 1, compare a permit view with the signed internal representation. The event ordering is the same; the displayed integers describe different bookkeeping.

Event	Available permits	Signed internal value
Initial	1	1
A completes wait	0; A proceeds	0; A proceeds
B attempts wait	0; B waits	-1; B is queued
A signals, passing permission to B	B obtains the returned permit; no unused permit	0; B is made eligible to resume
B eventually signals	1	1

The signed value -1 describes one recorded waiter. It does not mean a printer count has become physically negative. A just-awakened thread may still wait for CPU service; wakeup does not mean immediate execution.

19.7 Choose the abstraction by the required condition

Use a mutex for owned access to a shared invariant. Use a counting semaphore for a finite number of interchangeable permits. Use a semaphore initially zero when a signal must persist until a later wait. Use a mutex/monitor plus condition variable when entry depends on a predicate over shared state, such as “balance is sufficient” or “the queue is nonempty”.

A counting semaphore initialised to 3 intentionally allows three simultaneous permit holders. It does not enforce one-at-a-time access to unrelated buffer bookkeeping. Semaphores can coordinate correctly designed programs, but neither their presence nor their count alone prevents deadlock.

20 Monitors and condition variables

20.1 Monitor structure

A **monitor** encapsulates shared state and procedures with automatic mutual exclusion: at most one participant is active inside its protected procedures at a time. This reduces the burden of placing separate wait/signal lock operations around every access.

```
monitor SharedObject:
    private shared_state
    condition changed

    procedure operation():
        // Enter with the monitor's implicit lock held.
        while not predicate(shared_state):
            changed.wait()
        modify(shared_state)
        changed.signal()
```

The language/runtime or equivalent mutex-and-condition-variable arrangement must enforce the monitor rules; writing the word “monitor” above unsynchronised code is not an implementation.

20.2 Condition wait and signal

A **condition variable** lets a participant wait until a predicate over the protected state may be true. `cv.wait()` **atomically releases the associated lock and blocks**; before returning it reacquires that lock. Other participants can then enter the monitor and change the condition. Waiting without releasing the lock would prevent the very state change needed to wake the waiter.

`cv.signal()` wakes/notifies a waiting participant, if there is one. It does not store a permit for a future waiter. The actual durable information is the predicate in shared state. `broadcast` notifies all waiters, which still compete for the lock and recheck their predicates.

20.3 Hoare versus Mesa semantics; why a while loop is used

With **Hoare-style** signalling, control is immediately transferred to the awakened monitor participant; the signalling participant temporarily yields the monitor. With **Mesa-style** signalling, the signaller continues and the waiter becomes eligible to resume later after reacquiring the lock. By then another participant may have changed the state. Real interfaces may also permit spurious wakeups.

Therefore the robust Mesa-style pattern is **while predicate is false, wait**. A notification means "recheck", not "the condition is reserved for you". Test the predicate and call wait with the same associated lock held so the state check and release-and-sleep are correctly coordinated.

Worked example: Why an if guard can fail

Consumer A waits for a nonempty buffer. A producer adds one item and signals. Before A reacquires the monitor, consumer B enters and removes that item. If A had used `if empty then wait`, it would resume and remove from an empty buffer. With `while empty: wait`, it rechecks and waits again.

20.4 Condition variable versus semaphore

Feature	Condition variable	Semaphore
Durable count of past signals	No stored permit	Permits can persist until consumed
Meaning of wait	Release lock, sleep, reacquire lock	Obtain a permit or wait for one
Required state logic	Predicate under associated lock	Correct permit accounting/protocol
Signal with no waiter	No queued notification for a future waiter	Can increase available permits

A condition is a predicate, not the variable's name

`notEmpty` is a wait queue/condition variable. The condition actually required is `count > 0`. A signal is useful only together with correctly maintained shared state and the corresponding guarded wait.

Further reading: OSC 9e, Chapter 5 monitors; OSTEP Chapter 30.

20.5 A monitor that combines exclusion and a predicate

Consider a shared account with a balance initially zero. A withdrawal must wait until enough funds exist, while deposits make that predicate more likely to become true.

```
monitor Account:
    balance = 0
    condition fundsChanged

    procedure deposit(amount):
        require amount > 0
        balance = balance + amount
        fundsChanged.broadcast()

    procedure withdraw(amount):
        require amount > 0
        while balance < amount:
            fundsChanged.wait()
        balance = balance - amount
```

The implicit lock prevents lost balance updates. Waiting releases that lock, allowing a deposit to occur. Broadcast lets several waiters recheck possibly different requested amounts; each reacquires the lock before checking and subtracting. A withdrawal that wakes cannot rely on the earlier deposit still being available after other withdrawals.

This teaching example omits overflow and application-specific policies. It protects the balance and availability condition, but does not guarantee fair selection among waiting withdrawal amounts. Exclusion, condition satisfaction and fairness are separate responsibilities.

21 Producer–consumer: the bounded-buffer problem

21.1 Problem and requirements

Producers generate items; consumers use them. A shared buffer holds at most N items. A correct solution must protect the buffer and its indices, make a producer wait when no slot is available and make a consumer wait when no item is available. Producers and consumers should perform their unrelated work outside the buffer's protected section.

A circular buffer can use `in` for the next insertion position and `out` for the next removal position. Advance either index modulo N. These indices and the buffer must be accessed consistently; their exact representation is separate from synchronization.

21.2 Semaphore solution: complete initialisation and code

Use counting semaphores `empty=N`, `full=0`, and a binary exclusion semaphore `mutex=1`.

```
Producer:
    repeat:
        item = produce_item()
        wait(empty)             // Reserve a free slot.
        wait(mutex)             // Enter buffer critical section.
        buffer[in] = item
        in = (in + 1) mod N
        signal(mutex)
        signal(full)            // Publish one available item.

Consumer:
    repeat:
        wait(full)              // Reserve an available item.
        wait(mutex)
        item = buffer[out]
        out = (out + 1) mod N
        signal(mutex)
        signal(empty)           // Publish one free slot.
        consume_item(item)
```

Why each semaphore is necessary. `mutex` prevents conflicting buffer/index modifications. `empty` prevents insertion without capacity. `full` prevents removal without an item. A single mutex would prevent simultaneous access but would not by itself express "wait until an item/slot exists".

Why produce/consume are outside the CS. They may take substantial time and normally do not require exclusive access to the shared buffer. Holding the mutex during them would unnecessarily serialize independent work.

21.3 Worked count trace and the in-flight nuance

Worked example: Buffer capacity 3, completed operations only

Start with $(empty, full) = (3, 0)$ and no items. Two completed producer operations give $(1, 2)$ and two items. A completed consumer operation gives $(2, 1)$ and one item. Three completed consumer operations cannot occur now without further production: after consuming that last item, the next consumer blocks on `full`.

At these quiescent points, `empty+full=N`. But this is **not an invariant at every instruction**: a producer may already have decremented `empty` and not yet signalled `full`. Its slot is reserved in the meantime.

Let R_P be the number of producers that obtained `empty` but have not yet signalled `full`, and R_C the number of consumers that obtained `full` but have not yet signalled `empty`. Under the available-permit convention, the reservation invariant is

$$empty + full + R_P + R_C = N.$$

The number of physically stored items also need not equal `full` during an operation: an inserted item may not have been published, or a consumer may have reserved an item but not yet removed it.

Worked example: A reserved slot

Initially $N = 3$, `empty=3`, `full=0`. After one producer completes `wait(empty)`, the values are 2 and 0 with $R_P = 1$. The buffer may still contain zero items. After insertion but before `signal(full)`, there is one physical item and still zero published item permits. On publication, `full=1` and $R_P = 0$.

21.4 The dangerous reversed wait order

Suppose a consumer uses `wait(mutex)` before `wait(full)`. With an empty buffer, it holds the mutex and waits for an item. A producer can reserve an empty slot but cannot acquire the mutex to insert and publish that item. Both are stuck. Similarly, a producer holding the mutex while waiting for space in a full buffer prevents the consumer from creating space.

The required order is **wait for availability first, then enter the buffer CS**. Correct lock order is part of the solution, not a stylistic choice.

21.5 Monitor solution

Use protected state `count=0`, `in=0`, `out=0`; condition variables `notFull` and `notEmpty`. Every procedure below runs with the monitor lock held.

```
monitor BoundedBuffer:
    buffer[N]
    count = 0; in = 0; out = 0
    condition notFull, notEmpty

    procedure put(item):
        while count == N:
            notFull.wait()
        buffer[in] = item
        in = (in + 1) mod N
        count = count + 1
        notEmpty.signal()

    procedure get():
        while count == 0:
            notEmpty.wait()
        item = buffer[out]
        out = (out + 1) mod N
        count = count - 1
        notFull.signal()
        return item

Producer:                       Consumer:
    repeat:                         repeat:
        item = produce_item()           item = BoundedBuffer.get()
        BoundedBuffer.put(item)         consume_item(item)
```

The lock ensures exclusion; `count` expresses the predicates; condition waits release the lock so other participants can change them. The while loops recheck after reacquiring. No initial “permit count” is assigned to the condition variables.

What to include in a full bounded-buffer answer

State the capacity and requirements, give all three semaphore initial values, write both loops and explain the wait order. If asked for a monitor, use protected count/indices, two conditions, while guards and release-and-reacquire wait semantics. Explain overflow, underflow and exclusion separately.

Further reading: OSC 9e, Chapter 5 classical synchronization problems; OSTEP Chapters 30–31.

22 Readers–writers problem

22.1 Problem and policy choices

Many participants access a shared object. **Readers** only observe it; **writers** modify it. Multiple readers may access concurrently, but a writer must have exclusive access: no other writer and no reader may access during its write.

A single mutex around all reading/writing is safe but loses reader concurrency. The readers–writers problem asks for the more permissive protocol and requires a choice about admission policy.

Policy	Admission preference	Possible issue
Reader preference (first problem)	New readers can join an active reading group even while writers wait	Continuous arrivals can starve writers
Writer preference (second problem)	Once a writer is waiting, stop admitting additional readers ahead of it	Continuous writers can starve readers
Fair/no starvation policy	Regulate admission so neither category is indefinitely bypassed	Requires appropriate queues and scheduling assumptions

22.2 Reader-preference semaphore solution

Initialise `readCount=0`, `rmutex=1`, `resource=1`. `rmutex` protects the count; `resource` excludes a writer from the whole admitted reader group.

```
Reader:
    repeat:
        wait(rmutex)
        readCount = readCount + 1
        if readCount == 1:
            wait(resource)      // First reader reserves resource.
        signal(rmutex)

        read_shared_object()    // Readers may run concurrently.

        wait(rmutex)
        readCount = readCount - 1
        if readCount == 0:
            signal(resource)    // Last reader releases resource.
        signal(rmutex)

Writer:
    repeat:
        wait(resource)
        write_shared_object()
        signal(resource)
```

First reader: claims the shared resource on behalf of the group. **Subsequent readers:** only update membership and can read alongside existing readers. **Last reader:** returns the group's resource permit. `readCount` tracks admitted reader membership, not necessarily the number physically executing a read instruction at that instant.

The first reader may block on `resource` while holding `rmutex`. This is safe for this particular protocol: the writer holding `resource` does not need `rmutex` to finish, and no already-admitted reading group needs to exit when the count has just changed from 0 to 1. Do not turn the bounded-buffer wait-order rule into a universal claim that waiting while holding any semaphore is always wrong; analyse dependencies.

Worked example: Reader group and a waiting writer

Reader A enters, taking `resource`; `readCount=1`. Reader B joins and the count becomes 2 without taking another resource permit. Writer W waits on `resource`. A finishes, leaving count 1; W still cannot write. B finishes, leaving count 0 and signalling `resource`, so W can acquire it. If new readers continually keep the count positive, W can remain blocked: this is writer starvation.

22.3 A fair-admission turnstile solution

A common supporting solution adds `turnstile=1`. Readers pass through it before joining; a writer holds it while waiting for and using the resource, preventing later readers from extending the current reading group indefinitely.

```
// Initial: turnstile=1, rmutex=1, resource=1, readCount=0.
Reader:
    wait(turnstile)
    signal(turnstile)

    wait(rmutex)
    readCount = readCount + 1
    if readCount == 1:
        wait(resource)
    signal(rmutex)

    read_shared_object()

    wait(rmutex)
    readCount = readCount - 1
    if readCount == 0:
        signal(resource)
    signal(rmutex)

Writer:
    wait(turnstile)
    wait(resource)
    write_shared_object()
    signal(resource)
    signal(turnstile)
```

While a writer owns the turnstile and waits for existing readers to finish, new readers cannot pass it. Once those readers exit, the writer obtains the resource. Readers that already passed the gate may still join; this code does not promise exact arrival-order service for each read instruction.

A no-starvation claim needs fair semaphore queues/acquisition and scheduling, and finite read/write sections. A semaphore specified only as an atomic count need not select waiters fairly. The extra gate prevents the reader-stream failure of the basic solution, but code alone does not establish a universal time bound.

22.4 Writer-preference monitor variant

This variant makes writer preference explicit. It is useful for comparing the two classical policy choices. State: `activeReaders=0`, `waitingWriters=0`, `writing=false`; conditions `canRead`, `canWrite`. Procedures execute exclusively inside the monitor; the actual data access happens between begin/end calls.

```
monitor RWController:
    activeReaders = 0
    waitingWriters = 0
    writing = false
    condition canRead, canWrite

    procedure beginRead():
        while writing or waitingWriters > 0:
            canRead.wait()
        activeReaders = activeReaders + 1

    procedure endRead():
        activeReaders = activeReaders - 1
        if activeReaders == 0:
            canWrite.signal()

    procedure beginWrite():
        waitingWriters = waitingWriters + 1
        while writing or activeReaders > 0:
            canWrite.wait()
        waitingWriters = waitingWriters - 1
        writing = true

    procedure endWrite():
        writing = false
        if waitingWriters > 0:
            canWrite.signal()
        else:
            canRead.broadcast()
```

A reader calls `beginRead()`, reads, then `endRead()`; a writer uses the analogous pair. All callers must reliably execute their end procedure after admission. Once a writer announces itself, new readers wait, even if no writer is currently active. This avoids a continuously growing reader group, but a continuous writer queue can delay readers indefinitely. Mesa while guards handle competition after wakeup.

Explain both safety and policy

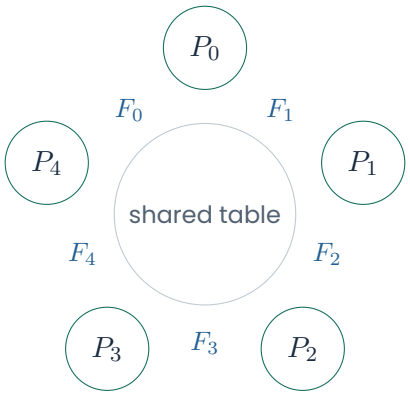
Safety is "many readers or one writer". Reader preference, writer preference and fairness concern *which waiting requests are admitted next*. A solution may satisfy safety and still starve a category. Name the policy your code implements.

Further reading: OSC 9e, Chapter 5 readers–writers; OSTEP Chapter 31.

23 Dining philosophers

23.1 Problem model and the naive failure

Five philosophers sit around a table. Each alternates between thinking and eating. There is one fork between each neighbouring pair, and a philosopher needs both adjacent forks to eat. Let philosopher i use fork i and fork $(i + 1) \bmod 5$. Each fork has a binary semaphore initialised to 1.



Fork F_i lies between P_{i-1} and P_i, with indices modulo 5; the labels match the code. Non-neighbouring philosophers may eat concurrently. At most $\lfloor 5/2 \rfloor = 2$ can eat at once because each needs two distinct forks and neighbours cannot share a fork while eating.

```
// Naive protocol: can deadlock.
wait(fork[i])
wait(fork[(i + 1) mod 5])
eat()
signal(fork[(i + 1) mod 5])
signal(fork[i])
```

If all five take fork i first, each holds one fork and waits for the next fork held by a neighbour. None can reach release: a circular wait produces deadlock. Individual fork exclusion does not solve the complete problem.

23.2 Solution 1: admit at most four contenders

Initialise `room=4` as a counting semaphore and all `fork[k]=1`.

```
Philosopher i:
    repeat:
        think()
        wait(room)
        wait(fork[i])
        wait(fork[(i + 1) mod 5])
        eat()
        signal(fork[(i + 1) mod 5])
        signal(fork[i])
        signal(room)
```

At most four philosophers attempt the two-fork acquisition simultaneously. The full five-way cycle cannot form; a gap in the contender circle allows someone to obtain both forks and eventually release them. Four is the *admission* limit, not the number that can eat at once. Fork ownership still limits simultaneous eaters to two.

This breaks the naive deadlock pattern under finite-meal/progress assumptions. It does not guarantee starvation freedom with unfair semaphore selection. The script checks 15,712 states for one meal per philosopher in this protocol, finding no nonterminal dead end and no adjacent eating pair; it is not an infinite-execution fairness proof.

23.3 Solution 2: globally order the forks

Assign each fork a unique index; require everyone to acquire the lower-numbered one first:

```
a = min(i, (i + 1) mod 5)
b = max(i, (i + 1) mod 5)
wait(fork[a])
wait(fork[b])
eat()
signal(fork[b])
signal(fork[a])
```

A participant holding one fork can wait only for a higher-numbered fork. A cycle would require fork indices to increase strictly and then return to the starting index, which is impossible. This removes circular wait. For philosopher 4, the order is fork 0 then fork 4; applying a consistent global ordering is more precise than saying “everyone picks up the left fork first”.

Asymmetric acquisition is another supporting option: one philosopher takes the adjacent forks in the opposite order to the others, breaking the common acquisition cycle. Mutual exclusion, deadlock freedom and starvation freedom remain separate questions.

23.4 Solution 3: monitor and neighbour state tests

Use state values `THINKING`, `HUNGRY`, `EATING`, initially THINKING for all five, and one condition variable `self[i]` per philosopher. The monitor chooses when a hungry philosopher has exclusive permission to eat.

```
monitor Dining:
    state[5] = THINKING
    condition self[5]

    procedure test(i):
        left  = (i + 4) mod 5
        right = (i + 1) mod 5
        if state[i] == HUNGRY and
           state[left] != EATING and state[right] != EATING:
            state[i] = EATING
            self[i].signal()

    procedure pickup(i):
        state[i] = HUNGRY
        test(i)
        while state[i] != EATING:
            self[i].wait()

    procedure putdown(i):
        state[i] = THINKING
        test((i + 4) mod 5)
        test((i + 1) mod 5)

Philosopher i:
    repeat:
        think()
        Dining.pickup(i)
        eat()                    // Outside the monitor.
        Dining.putdown(i)
```

Why exclusion holds. Only an exclusive monitor procedure can change a state to EATING, and it checks both neighbours first. EATING represents *reserved permission*, including the period before

a woken philosopher actually starts eating. This prevents a newcomer from stealing the permission before the waiter resumes.

Why the first signal can be harmless. If `test(i)` grants immediate permission during `pickup(i)`, no waiter may yet exist on `self[i]`. That signal is not stored, but the durable state is already EATING, so the while guard skips the wait. The shared state, not a remembered notification, carries permission.

Why wait releases the monitor. Otherwise no neighbour could call `putdown` to make progress. `putdown` tests both newly eligible neighbours when resources become available. The while guard is suitable for Mesa-style wakeups and rechecks state.

The solution prevents adjacent eating and the naive all-hold-one deadlock, but does not automatically ensure that a particular hungry philosopher is eventually preferred over all competing neighbours. A fairness policy would be needed to guarantee starvation freedom.

A strong dining-philosophers answer

Draw/define the ring and fork indices, show the all-take-one deadlock, give a complete remedy with initial values or monitor state, and explain why it works. Do not claim "no starvation" merely because the deadlock cycle is broken.

Further reading: OSC 9e, Chapter 5 dining philosophers; OSTEP Chapter 31.

23.5 Recognise the three different resource patterns

The bounded buffer has **two availability conditions**: a producer needs space, a consumer needs an item. Readers–writers has **two access classes**: many compatible readers or one exclusive writer. Dining philosophers has **overlapping multi-resource claims**: each task needs two neighbouring resources together. Understanding the pattern helps reconstruct the code instead of memorising unrelated wait/signal lines.

Problem	Safety rule	Liveness question
Bounded buffer	No overflow/underflow; coordinated item/index updates	Can the participant that creates the awaited condition obtain the lock?
Readers–writers	Readers can overlap; writer excludes all other access	Can the chosen preference keep the other category waiting forever?
Dining philosophers	Adjacent users do not reserve the same fork	Can two-resource acquisition create a cycle or indefinitely bypass a hungry neighbour?

23.6 Why a correct-looking solution may still starve someone

In the dining monitor, philosopher 0 needs neighbours 4 and 1 to be non-EATING together. Those two neighbours are not adjacent to each other and can alternate or overlap their meals. An unfair admission sequence can keep at least one of them reserved for eating whenever 0 is tested. Other philosophers keep progressing, so this is starvation rather than the naive all-hold-one deadlock.

Likewise, in reader preference, a writer can wait while a continuously overlapping reading group remains nonempty. Counting progress anywhere in the system is insufficient; test whether each announced waiter has an eventual admission guarantee.

24 Exam revision: concise answers

These questions are constructed from the syllabus, not predictions of an exam. The answers model the precision useful in a 3-mark response; adapt length to the actual question.

24.1 Module 1: short-answer practice

M1.1 Define an OS and its two central roles.

Answer. An OS manages hardware resources and provides services/abstractions to programs. As a resource manager it allocates CPU, memory, storage and devices; as a control program it supervises execution and prevents improper interference.

M1.2 State six OS functions.

Answer. Process/CPU management, memory management, file management, device/I/O management, communication and protection/security. Execution, error handling, storage allocation and accounting are further functions. Add examples if asked to explain.

M1.3 What is bootstrapping?

Answer. Bootstrapping brings the computer from reset to an executing OS. Firmware initialises essential hardware and selects a target; a loader brings the kernel into memory. The kernel initialises its subsystems and starts the user-space environment and services.

M1.4 Kernel versus shell?

Answer. The kernel is the privileged core managing execution and resources. A shell interprets commands and invokes programs/services, commonly in user space. A command interface is not the entire OS.

M1.5 Program versus process?

Answer. A program is passive stored instructions; a process is an executing instance with PC/registers, address space, resources and PID. Two processes can run the same program with separate execution state.

M1.6 Name the five process states.

Answer. New: being created. Ready: eligible but awaiting CPU. Running: executing. Waiting: unable to proceed before an event. Terminated: execution ended. I/O completion makes a waiting task ready; dispatch makes it running.

M1.7 What is a PCB? Give four fields.

Answer. The OS structure for managing and resuming a process. It records the PID and state, PC and registers, scheduling attributes, memory mappings and file or I/O references.

M1.8 What is a context switch?

Answer. The OS saves one execution context and restores another, including PC/registers and appropriate management state. It enables CPU sharing but has overhead. A system call can change mode without switching processes.

M1.9 What do threads share, and what is private?

Answer. Threads in a process normally share code, address space, globals, heap and open-file resources. Each has a PC, registers, stack and thread identifier. Shared writable state requires synchronization.

M1.10 State multithreading benefits.

Answer. Responsiveness, convenient resource sharing, often lower creation/switching cost than separate processes, and possible multicore parallelism. Actual gains depend on workload, mapping and contention.

M1.11 Compare the three schedulers.

Answer. Long-term admission selects jobs for the active workload; short-term scheduling chooses a ready task for CPU service; medium-term scheduling suspends/resumes tasks in systems that use it. The dispatcher implements the handover.

M1.12 Fork versus exec?

Answer. Successful fork creates a child, returning 0 there and the child PID in the parent. Exec replaces the current program image, creates no extra process and does not return to the old program on success.

M1.13 Zombie versus orphan?

Answer. A zombie has terminated but retains status awaiting collection. An orphan is a live child whose parent terminated and may be adopted by a reaper. An orphan is not necessarily terminated or a zombie.

M1.14 Explain shared-memory IPC.

Answer. The OS establishes a region mapped by participating processes. They exchange data by accessing it and coordinate conflicting accesses with synchronization. Mapping alone does not prevent races.

M1.15 Direct versus indirect messages?

Answer. Direct communication explicitly names another participant. Indirect communication uses a mailbox/port/channel shared by senders and receivers. Delivery and blocking rules depend on the interface.

M1.16 Compare message-buffer capacities.

Answer. Zero capacity holds no pending messages and commonly requires rendezvous. Bounded capacity holds up to N, requiring waiting/error handling when full. Theoretical unbounded capacity never fills; real systems are finite and other blocking semantics can still apply.

24.2 Module 2: short-answer practice

M2.1 Preemptive versus nonpreemptive scheduling?

Answer. A preemptive policy can remove the CPU from a runnable task. A nonpreemptive policy lets it continue until finishing or blocking/relinquishing. SRTF/RR are preemptive; ordinary FCFS/SJF are nonpreemptive.

M2.2 Define turnaround, waiting and response time.

Answer. TAT = CT − AT. WT is total ready-queue time; in the single-burst zero-overhead model WT = TAT − BT. RT = first start − AT for the standard numerical model.

M2.3 What is the convoy effect?

Answer. A long CPU-bound task holds up shorter/I/O-oriented tasks under FCFS, producing long waits and potentially inefficient CPU/device activity across the workload. FCFS is insensitive to burst length.

M2.4 SJF versus SRTF?

Answer. Nonpreemptive SJF selects the shortest ready next burst at dispatch and lets it finish. SRTF compares remaining service and can preempt for a newly ready shorter job. Neither can select a job before arrival.

M2.5 What is ageing?

Answer. Improve a waiting task's priority over time to counter indefinite postponement. If smaller values mean higher priority, decrease its number according to policy. Ageing differs from priority inheritance.

M2.6 Effect of a large or small RR quantum?

Answer. If every next burst fits a turn, RR behaves like FCFS for that workload. A very small quantum gives frequent turns but increases switching overhead. Balance responsiveness and cost.

M2.7 Define a race condition.

Answer. Correctness/result depends on uncontrolled concurrent access timing. Two read–modify–write updates can overwrite each other. Protect the whole shared operation or use correctly ordered atomic mechanisms.

M2.8 State the three CS requirements.

Answer. Mutual exclusion prevents simultaneous protected entry. Progress avoids indefinite selection delay when the CS is free and interested participants exist. Bounded waiting limits the number of other entries overtaking an announced request.

M2.9 Why does strict alternation fail?

Answer. It can force a participant to wait for someone remaining in its remainder section even though the CS is free. It can satisfy exclusion while violating progress.

M2.10 Flag and turn in Peterson's solution?

Answer. Flag announces interest; turn resolves competition. Process i sets its flag, gives turn to j and waits while j is interested and favoured. The proof requires atomic, sequentially consistent accesses.

M2.11 Why is local interrupt disabling insufficient on several cores?

Answer. It can prevent local interrupt-driven preemption, but another core can still access the resource. Cross-core exclusion requires an appropriate atomic lock/protocol. Ordinary user programs also cannot generally disable interrupts.

M2.12 Explain test-and-set.

Answer. It atomically returns the old lock value and sets it true. One contender can observe false and acquire it. A simple spin loop consumes CPU while waiting and does not guarantee bounded waiting.

M2.13 Spinlock versus blocking mutex?

Answer. Spinning repeatedly checks/attempts while remaining runnable. Blocking suspends a waiter and resumes it later. Spinning may suit short waits on another executing core; blocking avoids CPU consumption during long waits.

M2.14 What is a counting semaphore?

Answer. It represents permits manipulated through atomic wait/signal. Initialise to N for N units; wait obtains one and signal returns one. An initial 0 can instead implement event ordering.

M2.15 Why is a binary semaphore not identical to a mutex?

Answer. A mutex normally has an owner that must unlock. A semaphore can be signalled by another participant and represent events/availability. API, ownership and fairness semantics differ even when either supports exclusion.

M2.16 What does a condition wait do?

Answer. Atomically releases its associated lock and blocks, reacquiring before returning. The caller rechecks the shared predicate in a while loop. Condition signals do not store future permits.

M2.17 Bounded-buffer initial values?

Answer. For N slots, empty $= N$, full $= 0$, mutex $= 1$. Producers wait empty then mutex; consumers wait full then mutex. Waiting for availability before locking avoids the full/empty lock-holding deadlock.

M2.18 Can empty + full be below N in a correct solution?

Answer. Yes, while operations have reserved permits and not yet signalled the corresponding availability. At completed-operation quiescent points the sum is N; generally include producer and consumer reservations.

M2.19 Why can readers starve a writer?

Answer. In reader preference, new readers can join an existing group. Continuous arrivals can keep readCount positive, retaining the group's resource permit and blocking a writer indefinitely.

M2.20 Show dining-philosophers deadlock.

Answer. All five hold one fork and wait for the next, held by a neighbour. None can release. A room limit of four or a global fork-acquisition order breaks this circular wait.

M2.21 Deadlock freedom versus starvation freedom?

Answer. Deadlock freedom excludes a set permanently blocking each other through dependencies. Starvation freedom requires every eligible waiter eventually to progress. Others can keep progressing while one starves; a deadlock-free protocol need not be fair.

25 Long-answer practice and worked solutions

For a 9-mark explanation, define the problem, develop the mechanism and explain consequences/limits. Include requested diagrams, complete code or calculations. These are answer structures, not an official marking scheme.

25.1 OS functions and the complete boot sequence

Define the OS; explain CPU/process, memory, file/storage, device/I/O, communication, protection and error-handling functions with examples. Draw and explain the sequence firmware → loader → kernel initialisation → root file system → user-space services/login. Identify which component locates/loads the kernel and which starts the user environment. Use Sections 1–2 for the expanded answer; vendor names alone are insufficient.

25.2 Process states, PCB and context switching

Draw the five-state diagram from Section 3.2, label every transition and define each state. Explain PCB fields from Section 3.3. Trace running → I/O wait → ready → later dispatch. Explain saving/restoring context and its cost. Distinguish ready from waiting and mode changes from process changes.

25.3 Process operations and threads

Explain creation identity/resources, parent–child address-space choices, concurrent execution and termination cleanup. Illustrate fork/exec/wait/exit with correct return values; distinguish zombies and orphans. For threads, describe shared/private resources, benefits and mapping models. If the question asks only one part, devote the answer to it rather than forcing unrelated material into it.

25.4 Both IPC models and message semantics

Draw two processes and either a shared region or a channel/mailbox. Explain mapping plus explicit synchronization versus interface-mediated transfer. Compare setup, isolation, transfer cost and distribution. Explain direct/indirect naming, blocking/nonblocking operations and zero/bounded/unbounded capacity; distinguish completion semantics from capacity.

25.5 Fresh numerical problem: FCFS, SJF and RR

Schedule these single-burst jobs using FCFS, nonpreemptive SJF and RR with $q = 2$. Assume zero overhead and arrivals before expired-process requeue.

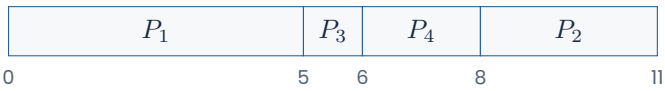
Process	Arrival	Burst
P_1	0	5
P_2	1	3
P_3	2	1
P_4	4	2

FCFS solution: use arrival order P_1, P_2, P_3, P_4.



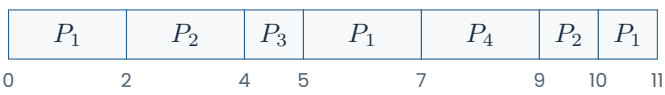
Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	5	0	5	5	0	0
P_2	1	3	5	8	7	4	4
P_3	2	1	8	9	7	6	6
P_4	4	2	9	11	7	5	5

Average turnaround $= 6.50$, waiting $= 3.75$ and response $= 3.75$. **SJF solution:** P_1 starts at 0 and cannot be interrupted. At 5 select ready bursts 1, 2 and 3 in that order.



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	5	0	5	5	0	0
P_2	1	3	8	11	10	7	7
P_3	2	1	5	6	4	3	3
P_4	4	2	6	8	4	2	2

Average turnaround $= 5.75$, waiting $= 3.00$ and response $= 3.00$. **RR solution:** at 2 the queue is $[P_2, P_3, P_1]$; at 4 it is $[P_3, P_1, P_4, P_2]$ because P_4 joins before P_2 is requeued.



Process	AT	BT	First	CT	TAT	WT	RT
P_1	0	5	0	11	11	6	0
P_2	1	3	2	10	9	6	1
P_3	2	1	4	5	3	2	2
P_4	4	2	7	9	5	3	3

Average turnaround $= 7.00$, waiting $= 4.25$ and response $= 1.50$. Total service is $5 + 3 + 1 + 2 = 11$, matching the final time with no idle interval. RR gives P_1 WT $= 11 - 0 - 5 = 6$ and RT $= 0$; interrupted waiting accounts for the difference.

25.6 Peterson's solution with correctness argument

State two-participant, atomic-access and sequential-consistency assumptions. Give complete entry/exit code, describe intent and yielding turn, and trace simultaneous interest. Address mutual exclusion, progress and bounded waiting separately using Section 16. End with busy-wait/two-participant limits and real-machine memory-order requirements.

25.7 Hardware locks, mutexes and semaphores

Define atomic TAS/CAS and give an acquisition/release loop. Explain indivisible acquisition, spinning cost and lack of guaranteed fairness. Compare spinning/blocking and mutex ownership/semaphore permits. Show A-before-B ordering with a semaphore initially 0. Explain why local interrupt disabling cannot give general cross-core exclusion.

25.8 Bounded buffer and a faulty wait order

Give both loops and initial empty $= N$, full $= 0$, mutex $= 1$. Explain capacity, published items and index protection. For faulty consumer "wait(mutex); wait(full)", start with an empty buffer: it retains the lock while waiting, so the producer cannot insert. The full-buffer/faulty-producer case is symmetric. If asked for a monitor, give count predicates, both conditions and while guards.

25.9 Readers–writers and policy

State many-readers/one-writer safety. Give reader-preference code and explain first-reader resource acquisition, count protection and last-reader release. Trace two readers and a waiting writer; show continuous readers can starve it. Compare writer preference and fair admission, stating queue/progress assumptions rather than asserting that all semaphores are fair.

25.10 Dining philosophers with two solutions

Define consistent fork indices, demonstrate all-take-one circular wait, then give room $= 4$ or globally ordered acquisition with all release steps. Explain which dependency cycle is removed. For a monitor, give state, neighbour tests, wait and putdown logic. Discuss starvation separately; EATING state denotes reserved permission.

25.11 Semaphore ordering and a Mesa wait trace

With done $= 0$, B arriving at wait first blocks until A performs its action and signals. If A signals first, its permit persists until B waits. A condition signal with no waiter stores no notification. A monitor checks a durable predicate under its lock; if false, wait atomically releases/sleeps, later reacquires and rechecks. Mesa competition or spurious wakeups make an if-only guard unreliable. These different traces demonstrate why semaphore wait and condition wait are not interchangeable.

26 Final syllabus checklist and common mistakes

26.1 Check these after active recall

Module	Explain without looking	Produce the required artefact
1	OS overview/functions	Layers and service examples
1	Boot process	Firmware–loader–kernel–services sequence
1	States and PCB	State diagram, transitions, PCB fields
1	Threads and scheduling	Shared/private resources; scheduler versus dispatcher
1	Creation and termination	Fork/exec/wait trace; zombie/orphan distinction
1	Both IPC models	Naming, blocking and buffering choices
2	Scheduling basics/criteria	Preemption decisions and metric formulas
2	FCFS, SJF, priority, RR	Gantt charts, timing tables and averages
2	Races and CS problem	Lost update; three requirements
2	Peterson	Full code, trace and proof
2	Hardware and mutex locks	TAS/CAS, spinning/blocking, ownership
2	Semaphores and monitors	Counts, ordering and condition waits
2	Producer–consumer	Full loops and correct wait order
2	Readers–writers	First/last-reader logic and preference policies
2	Dining philosophers	Deadlock trace and complete justified remedy

26.2 Mistakes that change the answer

- Selecting an unarrived process, omitting idle time or using original instead of remaining bursts in SRTF.
- Leaving the priority convention unstated or changing arrival/requeue tie rules midway.
- Equating WT and RT in preemptive schedules or treating first interruption as completion.
- Moving Waiting directly to Running; confusing shell/kernel or exec/process creation.
- Assuming shared memory supplies exclusion or is always fastest.
- Claiming strict alternation ensures progress, simple TAS ensures fairness, or volatile makes Peterson universally correct.
- Treating condition wait as semaphore wait, or replacing Mesa while guards with if.
- Assuming empty + full = N at every instruction, or holding the buffer mutex while waiting for availability.
- Serialising all readers, or claiming that reader preference cannot starve writers.
- Confusing room $= 4$ with four eaters; inferring starvation freedom from deadlock freedom.

27 Sources and recommended study resources

27.1 Use a small, purposeful set of resources

The syllabus controls scope. Use this guide as the main study sequence, the prescribed textbook/official materials for terminology, OSTEP for another explanation and web pages for extra practice. Other courses' module boundaries can differ.

Prescribed book: Silberschatz, Galvin and Gagne, *Operating System Concepts*, 9th edition, Wiley India, 2015, as printed in the syllabus. The official chapter slides were consulted for Chapters 1–6; they are not republished in the bundle.

OSC 9e	Chapter	Read for
1	Introduction	OS roles, resources, modes, bootstrap
2	Operating-System Structures	Selected services/system-call and system-boot sections
3	Processes	States, PCB, scheduling roles, lifecycle, IPC
4	Threads	Sharing, benefits, user/kernel threads, mapping
5	Process Synchronization	CS, Peterson, hardware, mutexes, semaphores, monitors, classical problems
6	CPU Scheduling	Basics, criteria, FCFS, SJF/SRTF, priority, RR

Official OSC ninth-edition slide index. Select by title as well as number; later editions can rearrange chapters. Scheduling follows synchronization in this book, so its order differs from the course module order. Full architecture taxonomies in Chapter 2 are beyond the topics listed here.

27.2 OSTEP: free extended explanations

Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, on the authors' official book page. Read selectively:

Topic	Author-hosted chapter and purpose
Process abstraction	4: The Abstraction: The Process — state and mechanism/policy distinction
Creation APIs	5: Process API — fork, exec and wait examples
Scheduling	7: Scheduling: Introduction — turnaround/response tradeoffs
Concurrency motivation	26: Concurrency: An Introduction — threads and shared updates
Lock mechanisms	28: Locks — hardware, spinning/blocking
Condition waiting	30: Condition Variables — predicates, wakeups, buffers
Semaphore applications	31: Semaphores — ordering and classical problems

All these are direct links to the authors' chapter copies. Their deeper material is useful when needed; reading the entire book is unnecessary for these modules.

27.3 University notes: compact second explanations

John Bell's University of Illinois Chicago course notes reference OSC ninth-edition material. Relevant pages checked during preparation:

- Introduction: roles, organisation and environments.
- Processes: states, PCB, lifecycle and IPC.
- Threads: sharing, benefits and mappings.
- CPU Scheduling: policies, definitions and conventions.
- Synchronization: mechanisms and classical problems.

Use abbreviated notes critically: kernel entry also occurs through interrupts/exceptions, and a simple spinlock must not be assumed to provide bounded waiting.

27.4 Complementary textbook and practice websites

Tanenbaum and Bos, *Modern Operating Systems*, 4th edition, Chapters 1 (Introduction) and 2 (Processes and Threads), gives an alternative treatment of these topics. The publisher's contents identifies the chapter map. This is an optional pointer; a complete copy was not consulted.

The syllabus prints "6th Edition … 2015"; that combination conflicts with the publisher's 2015 fourth-edition listing. Use verified titles/chapter names to find a copy.

GeeksforGeeks offers short recaps and extra problems. Links were checked for relevance; primary textbook/author/university materials and independent calculations provided this guide's technical checks.

- CPU Scheduling: recap and links to individual algorithms.
- Priority Scheduling: extra examples; check priority conventions.
- Process Synchronization: a short review before full problem solutions.
- OS topic index: further examples; the syllabus remains the scope checklist.

27.5 Video options

NPTEL Operating System Fundamentals, Prof. Santanu Chattopadhyay, IIT Kharagpur, and NPTEL Introduction to Operating Systems, Prof. Chester Rebeiro, IIT Madras, provide slower explanations. Select process, scheduling and synchronization lectures. Course pages were checked; entire series were not reviewed.

The department syllabus also provides a Module 1 video and Module 2 video. These are preserved syllabus pointers; their full contents were not verification sources.

27.6 Reproducible verification record

Prepared against the local PEEOT521 syllabus on 7 October 2026. Prose, diagrams, questions and numerical examples are newly written. `verify_examples.py` recomputes six main schedule traces and three fresh-practice traces; the six main traces also have separately specified expected timelines.

The script enumerates lost-update interleavings and finite atomic-step models of Peterson, a two-slot buffer and room-of-four dining. They check the finite safety/deadlock cases described in the text, not infinite-execution fairness or compiled-machine memory ordering.

Both editions use identical content. The bundle includes LaTeX, fonts, generated tables, scripts, a validation record and a four-page syllabus extract. The black edition uses RGB $(0, 0, 0)$ for page and panel backgrounds. Final checks cover fonts, internal links/bookmarks, page bounds, matching study text and black backgrounds on every page.

28 GeeksforGeeks topic map and expanded lessons

The relevant GfG articles were compared topic by topic on 7 October 2026. The lesson text is independently written: these links provide a second explanation and practice, while the syllabus and textbook materials control scope and correctness. The source bundle contains the detailed change record.

Read the guide first; use an article for a second explanation

The expanded edition adds short introductory examples before larger problems, execution traces, mapping diagrams and advantages/limitations. You can study the mechanisms directly from the guide without having to assemble the explanations from websites.

28.1 Module 1: topic links

Topic and guide section	GeeksforGeeks article
OS overview — 1	Read the matching article
Functions and services — 1	Read the matching article
Types of operating system — 1	Read the matching article
Kernel and system calls — 1	Read the matching article
System-call execution — 1	Read the matching article
Boot process — 2	Read the matching article
Program versus process — 3	Read the matching article
Process states and transitions — 3.2	Read the matching article
PCB — 3.3	Read the matching article
Process table — 3.3	Read the matching article
Threads — 4	Read the matching article
User and kernel threads — 4	Read the matching article
Thread mapping models — 4	Read the matching article
Benefits of multithreading — 4	Read the matching article
Scheduling roles — 5	Read the matching article
Creation and termination — 6	Read the matching article
IPC, shared memory and messages — 7	Read the matching article

28.2 Module 2: topic links

Topic and guide section	GeeksforGeeks article
CPU scheduling basics and criteria — 8	Read the matching article
Preemption — 8	Read the matching article
Dispatcher versus scheduler — 8	Read the matching article
FCFS — 10	Read the matching article
SJF and SRTF — 11	Read the matching article
Priority scheduling — 12	Read the matching article
Starvation and ageing — 12	Read the matching article
Priority inversion — 12	Read the matching article
Round robin — 13	Read the matching article

Topic and guide section	GeeksforGeeks article
Synchronization and races — 15	Read the matching article
Race conditions — 15	Read the matching article
Critical-section requirements — 15.2	Read the matching article
Peterson solution — 16	Read the matching article
Synchronization hardware — 17	Read the matching article
Mutex locks — 18	Read the matching article
Semaphores — 19	Read the matching article
Mutex versus semaphore — 19	Read the matching article
Monitors and conditions — 20	Read the matching article
Producer-consumer — 21	Read the matching article
Readers-writers — 22	Read the matching article
Dining philosophers — 23	Read the matching article

The website also includes material for other courses. Supporting sections in this guide explain the named mechanisms; the GfG index is not a replacement syllabus.